



Lehrstuhl für Informatik

Multi-Agent 3D Coverage Path Planning Using a Hybrid Liquid Neural Reinforcement Learning Model for Autonomous UAV Systems

Masterarbeit von

Nauman Ahmed

PRÜFER

BETREUER

Prof. Dr. Mario Döller

Mohammadreza Mohebbi

Prof. Dr. Harald Kosch

March 31, 2026

Contents

Abstract	v
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Research Gap and Novelty	2
1.4 Research Questions	3
1.5 Contributions	3
1.6 Structure of the Thesis	4
2 Background	5
2.1 Coverage path planning	5
2.1.1 Problem definition	5
2.1.2 Classical CPP approaches	6
2.1.3 2D versus 3D CPP	8
2.1.4 Multi-agent CPP	9
2.2 Reinforcement learning and proximal policy optimisation	10
2.2.1 MDP and partial observability	10
2.2.2 Policy gradient methods	11
2.2.3 Proximal policy optimisation	11
2.2.4 PPO for CPP	13
2.3 Liquid neural networks	13
2.3.1 Biological inspiration (NCP)	13
2.3.2 Liquid time-constant networks (overview)	14
2.3.3 AutoNCP wiring and neural circuit policies	15
2.3.4 LTC versus LSTM and GRU: conceptual comparison	15
2.3.5 Prior applications of LTC in robotics	16
2.3.6 Why LTC suits CPP: conceptual match	17

2.4	Related work and research gap	17
2.4.1	Coverage path planning and reinforcement learning	18
2.4.2	Multi-agent reinforcement learning for coverage and navigation . .	19
2.4.3	Liquid time-constant networks and prior robotics use	19
2.4.4	Motivation for combining PPO, LTC, and multi-agent CPP . . .	20
2.4.5	Research gap and novelty of this thesis	20
3	Methods	22
3.1	Environment design	22
3.1.1	3D grid world	22
3.1.2	Action space, masking, and episode termination	28
3.1.3	Budget constraint	29
3.1.4	Episode seeding and environment diversity	30
3.2	Reward function	30
3.2.1	Reward design philosophy	30
3.2.2	Individual reward components	30
3.3	Policy architecture: LNN policy	34
3.3.1	Feature extraction pipeline	34
3.3.2	Liquid Time-Constant (LTC) recurrent core	35
3.3.3	Attention-based agent communication	38
3.3.4	Policy and value heads	38
3.4	Training and evaluation protocol	38
3.4.1	Training protocol	38
3.4.2	Four-checkpoint evaluation strategy	41
4	Results	42
4.1	Training dynamics	42
4.1.1	Coverage trend and end-of-training status	42
4.1.2	Distributional views of training coverage	44
4.1.3	Reward structure and training correlations	45
4.1.4	Training diagnostics and PPO metrics	47
4.2	Checkpoint evaluation (100 fixed seeds; greedy decoding)	53
4.2.1	Core checkpoint metrics	53
4.2.2	Evaluation thresholds and diagnostic summary	54
4.2.3	Greedy versus stochastic action selection (latest checkpoint) . . .	55
4.3	Overall summary	57

5	Discussion	59
5.1	Overview	59
5.2	RQ1: Can LTC–PPO learn effective 3D multi-agent CPP?	59
5.3	RQ2: Does attention-based LTC coordination produce safe cooperative behaviour?	61
5.4	RQ3: Which checkpoint criterion is most reliable?	62
5.5	RQ4: Is PPO training stable with LTC policies?	62
5.5.1	Role and importance of liquid neural networks in multi-agent coverage	63
5.6	Future work	65
5.7	Closing remarks	67
6	Conclusion	68
	Appendix A Software implementation and algorithms	69
A.1	Software stack	69
A.2	Modular structure	69
A.3	Target-area generation	69
A.4	Shortest-path distances and forced return (BFS)	70
A.5	High-level training procedure	71
	Appendix B Mathematical notation and supplementary formulas	73
B.1	Notation	73
B.2	Generalised advantage estimation	74
B.3	PPO clipped objective (scalar form)	74
	Appendix C Data artefacts, logs, and evaluation protocol	75
C.1	Training logs	75
C.2	Checkpoints	75
C.3	Evaluation suites	76
C.4	Figures and provenance	76
	Appendix Declaration on the Use of Artificial Intelligence Assistance	77
	Bibliography	78
	Eidesstattliche Erklärung	83

Abstract

This thesis studies constrained 3D multi-agent coverage path planning (CPP) for UAV-style agents under terrain, volumetric no-fly zones, and hard movement-budget safety constraints. The work addresses a specific research gap identified in the literature review: to the best of my knowledge, Liquid Neural Networks (including Liquid Time-Constant (LTC) models) have not been applied to coverage path planning as an end-to-end learned policy.

I formulate the task as a reinforcement learning problem and train an LTC-based recurrent policy with Proximal Policy Optimisation (PPO). The model combines multi-branch spatial feature extraction, an LTC recurrent core for temporal decision-making, and attention-based inter-agent communication. The environment and protocol are fully specified and reproducible, including procedural layout generation, legal-action masking, forced-return safety, fixed-seed evaluation suites, and checkpoint selection by multiple criteria.

Across **11,436** training episodes, coverage and reward improve consistently, with a strong positive association between return and final coverage. Late training windows show near **97%** mean coverage. On held-out fixed-seed evaluation, the checkpoint selected by the best 100-episode rolling mean of return is the most reliable among the four criteria tested, achieving approximately **93.0%** mean coverage at **budget 150** and **88.0% at budget 100**, with favourable threshold hit rates and exploration efficiency. Safety and coordination diagnostics are also consistent with zero collision penalties in fixed-seed evaluation and complementary exploration among agents.

Overall, the thesis provides feasibility evidence that an LTC-PPO pipeline can learn effective behaviour for constrained 3D multi-agent CPP and establishes a concrete baseline for future controlled comparisons, broader benchmarking, and scalability studies.

Acknowledgements

I am grateful to Mohammadreza Mohebbi for supervising this work, for constructive discussions on reinforcement learning and coverage path planning, and for feedback on drafts and experimental choices throughout the thesis.

I thank Prof. Dr. Mario Döller for academic guidance within the Chair of Informatik and for supporting the research direction behind this Master's thesis. I also thank Prof. Dr. Harald Kosch for serving as second examiner and for his time and careful reading of the work.

Several practical aspects of the project depended on reliable software and documentation. I acknowledge the developers of the scientific Python ecosystem, PyTorch, and related libraries that made implementation and reproducible experimentation feasible.

Finally, I owe my deepest gratitude to my family, whose unwavering support made this journey possible. I thank my mother, Samina Anwar, for her endless patience, love, and encouragement throughout my studies. I thank my father, Anwar Ahmed, for his steadfast belief in my abilities and for the sacrifices he made to support my education. I also thank my elder brother, Ahsan Ahmed, for his guidance and for always being a source of strength, and my younger brother, Arsalan Ahmed, for his cheerfulness and encouragement during the more demanding stages of this work. I extend my warm thanks to my sister-in-law, Huma Ahsan, for her kindness and moral support throughout this journey. This thesis is dedicated to all of them.

List of Figures

3.1	Environment view (3D). Terrain, the target-area surface layer, NFZ volumes, SLZ locations, and both agents are shown together.	25
3.2	Environment view (top-down). This view highlights the horizontal footprint of the TA and NFZ and the spatial relationship between SLZ spawns and the agents.	26
3.3	Environment view (terrain heatmap). Peaks and valleys explain why the same absolute altitude can be safe in one column but infeasible in another.	27
3.4	Policy architecture overview. The observation is processed by parallel feature branches (global statistics, local 3D and 2D views, scalar urgency, and goal/uncovered-direction features). The concatenated per-timestep features are passed through an LTC recurrent core (AutoNCP wiring) and then through multi-head attention for inter-agent communication. The network outputs one masked 6-action policy head per agent and a shared value head for PPO.	36
4.1	Coverage trend (100-episode rolling mean).	42
4.2	Coverage over the last 100 episodes.	43
4.3	Coverage by stage (boxplots; thirds of the full run).	44
4.4	Distribution of episode coverage (histogram).	44
4.5	Mean reward components over episodes.	45
4.6	Coverage versus total reward during training (Pearson $r = 0.947$).	46
4.7	Leading indicators versus episode (MA-50 and MA-200).	47
4.8	Diagnostic indicators (six panels versus episode).	48
4.9	PPO and training metrics versus episode.	49
4.10	Jointly explored space across performance groups. The plotted quantity is the union of unique visited 3D cells across both agents within an episode.	51
4.11	Inter-agent overlap ratio across performance groups, measured as the Jaccard overlap between the sets of visited cells of agent 0 and agent 1.	52

List of Figures

4.12 Greedy versus stochastic coverage histograms (latest checkpoint, budget 150; 50 seeds).	55
4.13 Greedy versus stochastic coverage histograms (latest checkpoint, budget 100; 50 seeds).	56

List of Tables

3.1	Reward component statistics (logged training run) and qualitative purpose.	33
3.2	Dimensions for feature branches and heads.	37
3.3	Final hyperparameters and architecture settings used in the reported experiments.	40
4.1	Trailing-window coverage summary (same run).	43
4.2	Coverage by stage (episode index split into early/mid/late thirds of the full run).	43
4.3	Checkpoint comparison (core metrics). Movement budget 100.	53
4.4	Checkpoint comparison (core metrics). Movement budget 150.	53
4.5	Movement budget 150: coverage thresholds and hit rates (evaluation, 100 seeds).	54
4.6	Movement budget 150: task-focus diagnostics (console evaluation, 100 seeds).	54
4.7	Movement budget 100: coverage thresholds and hit rates (console evaluation, 100 seeds).	54
4.8	Movement budget 100: task-focus diagnostics (evaluation, 100 seeds). . .	55
4.9	Greedy versus stochastic (latest checkpoint, budget 150; 50 episodes). . .	56
4.10	Task-focus diagnostics for greedy versus stochastic (latest checkpoint, budget 150; same 50-episode batch).	56
4.11	Greedy versus stochastic summary (latest checkpoint, budget 100; 100 seeds, histogram annotations).	57
4.12	Greedy versus stochastic (latest checkpoint, budget 100; 50 episodes). . .	57
4.13	Task-focus diagnostics for greedy versus stochastic (latest checkpoint, budget 100; same 50-episode batch).	57
B.1	Frequently used notation.	73

1 Introduction

1.1 Motivation

Coverage path planning (CPP) asks an autonomous system to move through an environment so that a target region is observed as completely and efficiently as possible [GC13; CBF19]. For UAV systems, this problem appears in inspection, monitoring, and search tasks where missing important regions can directly reduce mission quality [Lyu+23]. In practical deployments, CPP is rarely unconstrained: the agent must operate under limited energy or movement budget, avoid restricted space, and still maintain a safe return strategy.

These constraints become more challenging in 3D environments and in multi-agent settings. Terrain changes the effective flyable volume, no-fly zones remove feasible paths, and two agents must coordinate to avoid collisions and redundant overlap. Classical CPP methods remain valuable and often provide strong geometric guarantees in simplified settings [Cho01; GR01], but they are less straightforward to scale to highly variable procedural environments where constraints interact tightly and online adaptation is required.

This motivates a learning-based approach. Reinforcement learning (RL) offers a way to learn policies that map observations to actions under uncertainty [SB18; Mni+15; Sil+16], while multi-agent training can capture cooperative behaviour. For optimisation, Proximal Policy Optimisation (PPO) is a practical and widely used policy-gradient method for stable training of neural policies with large observation space [Sch+17; Mni+16]. For temporal modelling, this thesis investigates a Liquid Time-Constant (LTC) recurrent core, a member of the broader LiquidNN family, which is designed to process signals with mixed time scales [Has+21; Has+18].

1.2 Problem Statement

In this thesis, I study constrained 3D multi-agent CPP in a voxel environment with:

- terrain-constrained flyability,
- volumetric no-fly zones (NFZ),
- per-agent movement budget with forced-return safety,
- two agents that must coordinate under shared mission objectives.

The central question is whether an LTC-based PPO policy can learn effective, safe, and stable behaviour under this combination of constraints, and whether the learned behaviour transfers to held-out fixed-seed evaluations.

1.3 Research Gap and Novelty

Prior work covers important parts of the landscape: RL-based CPP for UAVs, multi-agent RL coordination, and liquid recurrent models in robotics and control. However, based on the reviewed literature in Chapter 2, these lines have not been combined in the form studied here. In particular, to the best of my knowledge, there is no published work that applies Liquid Neural Networks (including LTC) to coverage path planning as an end-to-end learned policy.

The novelty of this thesis is the integrated combination of:

- a constrained **3D multi-agent** CPP formulation,
- an **LTC-based** recurrent policy with **attention-based** inter-agent communication,
- **PPO**-based training with a fixed and reproducible rollout protocol,
- evaluation over fixed-seed suites across multiple checkpoint criteria and movement-budget settings.

1.4 Research Questions

To address the problem and gap above, I use four research questions:

1. **RQ1:** Can LTC-PPO learn effective 3D multi-agent CPP under terrain, NFZ, and budget constraints?
2. **RQ2:** Does the attention-based LTC policy produce safe and cooperative multi-agent behaviour?
3. **RQ3:** Which checkpoint selection criterion is most reliable in this stochastic CPP setting?
4. **RQ4:** Is PPO training stable when combined with the proposed LTC-based recurrent policy?

1.5 Contributions

The main contributions of this thesis are:

- A reproducible constrained 3D multi-agent CPP environment and protocol (procedural terrain/NFZ, action masking, forced-return safety, fixed evaluation seeds).
- An LTC-based multi-agent policy architecture with attention-based communication, trained with PPO and TBPTT.
- A detailed empirical study of training dynamics, checkpoint reliability, and evaluation behaviour under two movement-budget regimes.
- A discussion of limitations and concrete next steps for controlled architectural comparisons and stronger statistical validation.

1.6 Structure of the Thesis

The remainder of the thesis is organised as follows:

- **Chapter 2 (Background):** CPP foundations, RL/PPO principles, LiquidNN/LTC concepts, and the related-work gap.
- **Chapter 3 (Methods):** Environment design, reward function, policy architecture, and training/evaluation protocol.
- **Chapter 4 (Results):** Training behaviour, checkpoint comparison, coordination indicators, and greedy versus stochastic evaluation summaries.
- **Chapter 5 (Discussion):** Interpretation of findings by research question, limitations, and future work (Section 5.6).
- **Chapter 6 (Conclusion):** Concise summary of contributions and outcomes.

2 Background

2.1 Coverage path planning

2.1.1 Problem definition

Coverage path planning (CPP) is the problem of generating a path (or a set of paths in the multi-agent case) that enables an agent to **cover** a target region under physical and operational constraints. “Coverage” can mean visiting every cell in a discretised grid, sensing every part of an area, or achieving a prescribed observation level (for example, visibility-based coverage in 3D).

CPP problems are commonly evaluated using:

- **Coverage ratio**: the fraction of the target region that has been visited or observed.
- **Path length / time**: total distance travelled or number of steps taken.
- **Energy / budget consumption**: fuel, battery, or a hard movement budget.
- **Redundancy / revisit rate**: how much of the path repeats already-covered areas.

In this thesis, CPP is studied in a setting that is more constrained than standard planar coverage: the environment is **3D**, includes **terrain**, includes **restricted regions** (no-fly zones), uses a **limited movement budget**, and uses **two cooperating agents**. These properties make the problem a long-horizon sequential decision task where coverage quality must be traded against safety and budget feasibility.

2.1.2 Classical CPP approaches

CPP has a long history in robotics [GC13]. Classical methods aim to provide reliable coverage using geometric reasoning and graph search. I summarise the main families here, because they explain what a “good” coverage strategy looks like and why many approaches follow a sweep/partition pattern.

Decomposition-based coverage

A common idea is to decompose a workspace into smaller regions where a simple sweeping policy can guarantee coverage. The best-known example is **boustrophedon** coverage, where the agent sweeps back and forth (like mowing a lawn) while handling obstacles by splitting the free space into cells. The cell decomposition step turns a complex environment into a set of subproblems:

- inside each cell, the agent follows a simple systematic sweep that covers the cell efficiently,
- between cells, the agent transitions along a connectivity structure.

The strength of decomposition is that it is clear and dependable when the map is known [Cho01]. Its weakness is that it relies on clean geometry and typically assumes a 2D surface. Extending decomposition ideas to 3D volumes is possible, but the number of cells and the complexity of their boundaries grows quickly.

Graph-based coverage

Another family converts the environment into a graph and then plans a traversal that reaches all required nodes. In grid-based settings, the free space naturally forms a graph with 4- or 8-neighbour connections (2D) or 6-/26-neighbour connections (3D). Coverage then becomes a graph traversal problem.

A simple example is **spanning-tree coverage**, where a spanning tree is built over the free space and then traversed to ensure the robot visits nodes systematically [GR01]. Tree traversal has appealing properties:

- it gives a structured exploration order,

- it can be implemented with limited memory (follow parent/child relationships),
- it can be extended with heuristics that reduce revisits.

However, graph-based coverage still assumes a known map and can become expensive when the graph is very large (which is typical in 3D voxel worlds). If obstacles and restricted regions are generated online, the agent must re-plan frequently or accept suboptimal paths.

Frontier-based exploration (coverage as exploration)

In many robotics systems, “coverage” is approached as an **exploration** problem [Yam97]. The agent maintains a map of known and unknown space and repeatedly selects a **frontier** (the boundary between known and unknown) as the next goal. This style is popular because it naturally handles incremental sensing:

- the agent explores until no frontier remains,
- map updates happen online as new areas are observed,
- the method works with partial knowledge of the world.

Frontier methods usually require a good mapping pipeline and a planner to reach frontiers. They also tend to be greedy: selecting the nearest frontier can be locally sensible but globally inefficient under a hard budget.

Multi-robot partitioning and assignment

For multiple robots, a common classical approach is to divide the region into subregions and assign each robot one region. This can be done using:

- **Voronoi partitioning** (robots cover their nearest region),
- grid clustering,
- auction-based task assignment for subgoals.

The benefit is reduced redundancy: if each robot has an assigned area, overlap decreases. The cost is coordination complexity: partitions must be updated when obstacles block routes or when one robot finishes early.

Why classical methods are not enough for this thesis setting

Classical methods are strong when the environment model is stable and when planning constraints are easy to express. In this thesis, several factors make classical CPP less direct:

- the environment is procedurally generated and changes from episode to episode,
- restricted regions (NFZs) and terrain create complex 3D feasibility constraints,
- the observation and coverage are history dependent (coverage accumulates over time),
- the movement budget introduces a hard constraint that can invalidate “complete coverage” strategies.

These points motivate a learning-based approach where a policy can react online to local structure while still maintaining a sense of long-horizon progress [Aru+17].

2.1.3 2D versus 3D CPP

Many CPP discussions begin in 2D, where the agent moves on a plane and obstacles are planar. Moving to 3D changes the structure of the problem rather than simply adding an extra coordinate.

Key differences include:

- **State-space growth:** a 3D grid increases the number of possible positions and local configurations significantly, which affects both planning complexity and the learning sample complexity.
- **Altitude management:** the policy must decide not only where to move laterally but also when to change altitude, which can alter visibility and feasibility.
- **Terrain-relative movement:** feasibility depends on the terrain surface; an absolute altitude can be safe in one column and unsafe in another.
- **3D obstacle and NFZ handling:** restricted regions are volumes, not line segments; they can block line-of-sight and motion in altitude-dependent ways.

- **Collision checking complexity:** multi-agent conflict constraints are more complex in 3D, especially when agents move concurrently.
- **Representation challenges:** the policy must encode volumetric occupancy, target structure, and agent states in a way that supports local decisions and global progress.

In short, 3D CPP increases both planning difficulty and representation difficulty. The policy must reason about feasible motion in 3D, but also about what parts of the target region have already been covered and what remains. These factors motivate learning-based approaches that can map rich observations to actions without explicit global search over the entire 3D state space.

2.1.4 Multi-agent CPP

Adding multiple agents introduces coordination challenges that do not appear in single-agent CPP. Even if each agent can solve a local coverage problem, the team must avoid wasting budget on redundant coverage and must respect safety constraints.

Core multi-agent CPP concepts include:

- **Division of labour:** agents should allocate regions or subgoals so that coverage is complementary.
- **Redundant coverage:** overlap reduces efficiency under a fixed budget.
- **Credit assignment:** in cooperative settings, it is not always clear which agent’s action caused a team-level improvement.
- **Collision avoidance and interference:** agents share space and can conflict (same-cell conflicts, swaps, blocking).
- **Coordination under shared limits:** budgets, time limits, and shared objectives couple the agents’ decisions.

A common learning setup for cooperative multi-agent tasks is **centralised training with decentralised execution (CTDE)** [Can+21; Foe+18]. Under CTDE, training can use a shared objective and can condition learning on joint information, while execution produces per-agent actions in a distributed way. CTDE is a useful idea for coverage

because the **goal is shared** (team coverage), but the actions are individual (each agent moves on its own).

Multi-agent CPP also raises a practical question: what does “good coordination” look like? In simple terms, I expect coordinated agents to show three behaviours:

- they avoid being in the same place at the same time (safety and efficiency),
- they spread out to reduce redundant revisits,
- they still return safely before budget exhaustion.

Together, 3D structure and multi-agent coordination make CPP a demanding sequential decision problem, motivating adaptive learning-based methods.

2.2 Reinforcement learning and proximal policy optimisation

2.2.1 MDP and partial observability

Reinforcement learning is commonly formalised as a Markov Decision Process (MDP), defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition dynamics, R is the reward function, and γ is a discount factor.

An RL agent learns a **policy** $\pi(a \mid s)$ that maps states to actions. Two additional functions are central:

- **Value function:** $V^\pi(s) = \mathbb{E}_\pi[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s]$
- **Action-value function:** $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a]$

In many robotics tasks, the agent does not observe the full underlying state. CPP is often **partially observable** for two reasons. First, the agent’s sensors typically provide local views rather than a full map. Second, even if the environment map is known, the coverage progress is history dependent: the value of going to a location depends on whether it was already covered earlier in the episode. This is not captured by the current position alone. Deep recurrent value learning is a standard response to partial observability in RL [HS15].

Partial observability motivates policies with **memory**, typically implemented using recurrent networks or other history encoders. Memory lets the policy carry information such as “where I have been recently” and “how much progress is left” across time steps.

2.2.2 Policy gradient methods

Policy gradient methods optimise the policy parameters directly by ascending an estimate of expected return [Wil92]. The simplest formulation (REINFORCE) uses Monte Carlo returns and is unbiased but high variance. Actor–critic methods reduce variance by learning a value baseline (the critic) alongside the policy (the actor) [Lil+16].

A standard actor–critic quantity is the **advantage**, which measures how much better an action is compared to the state’s baseline:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

Policy gradient approaches are attractive for CPP-like tasks because:

- they naturally support **stochastic policies** during training,
- they can handle complex action preferences under shaped rewards,
- they integrate well with high-dimensional observations when combined with neural function approximation.

2.2.3 Proximal policy optimisation

Proximal policy optimisation (PPO) is a widely used on-policy actor–critic algorithm designed to stabilise policy gradient training [Sch+17]. PPO can be viewed as a trust-region-inspired method that discourages large policy updates, building on ideas from trust-region policy optimisation [Sch+15].

The core PPO objective uses a **clipped surrogate** loss. Let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ be the probability ratio between the new and old policies. The clipped objective is:

$$L^{\text{clip}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)].$$

2 Background

PPO is typically trained with a combined loss that includes:

- **policy loss** (the clipped objective above),
- **value loss** (to fit the critic, often MSE between predicted value and a return target),
- **entropy regularisation** (to encourage exploration and prevent premature collapse),
- **generalised advantage estimation (GAE)** [Sch+18] to reduce variance in advantage estimates.

To make the PPO objective more intuitive, I interpret the clipping rule in simple terms. The ratio $r_t(\theta)$ tells how much the new policy increases or decreases the probability of the action that was actually taken during data collection. If an action had a positive advantage (it worked well), PPO wants to increase its probability. But PPO does not want that increase to be too large in one update, because large updates can destabilise training. Clipping enforces that “do not change the policy too much” behaviour.

Two additional pieces are important in practice.

Value loss. PPO learns a value function to reduce variance and to compute advantages. If the value function is inaccurate, the advantage signal can be noisy and policy updates can become unstable. The value loss keeps the critic aligned with the return scale.

Entropy regularisation. Entropy encourages the policy to keep exploring rather than immediately collapsing to a single deterministic action choice. In CPP-like settings, exploration is important because the agent may need to discover good coverage patterns and escape local loops.

Most PPO implementations also use **generalised advantage estimation (GAE)** to reduce variance. GAE combines multi-step temporal difference errors in a weighted sum. The usual definition is:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.$$

Here, λ controls the bias-variance trade-off. A higher λ uses longer-horizon information (lower bias, higher variance), while a lower λ uses shorter-horizon information (higher

bias, lower variance). In long-horizon tasks like CPP, this trade-off is important because terminal coverage outcomes can be far from the decisions that caused them.

PPO is a reasonable choice for stochastic CPP environments because it is robust in practice, supports continuing exploration via entropy, and can learn under shaped multi-component rewards. Its on-policy nature is often acceptable when environment generation and simulation are fast enough to produce fresh rollouts.

2.2.4 PPO for CPP

PPO has been applied in related domains such as UAV navigation, obstacle avoidance, and coverage-like objectives, where the agent must make sequential decisions from structured observations [BD22; Wan+25; The+20; The+21; Bay+21]. PPO is often chosen in these areas because it provides stable training under noisy returns and because it supports stochastic policies.

However, many CPP-oriented PPO systems use standard policy architectures (feedforward CNN/MLP or conventional RNNs). This matters because CPP is strongly history dependent, and the quality of memory can change whether the agent keeps revisiting the same region or spreads out efficiently. This leaves an open question as to whether **liquid** recurrent policies, particularly LTC-based models, are effective for **3D multi-agent CPP** under combined terrain, NFZ, and budget constraints [Hu+23; Lun+24; Ali+25].

2.3 Liquid neural networks

This section introduces Liquid Neural Networks (LNNs) with a focus on Liquid Time-Constant (LTC) networks and Neural Circuit Policies (NCPs). The goal is to explain why this family of recurrent models is a meaningful architectural option for long-horizon decision problems under partial observability.

2.3.1 Biological inspiration (NCP)

Neural Circuit Policies were proposed as compact, structured recurrent networks inspired by biological neural circuits (often discussed in the context of *C. elegans*) [Lec+19].

The central motivation is that complex adaptive behaviour can emerge from **small, structured**, dynamical systems with strong inductive bias, rather than from dense recurrence alone.

In practice, this motivation appears in **wiring priors**: rather than learning a fully connected recurrent network, an NCP constrains recurrence to a sparse pattern intended to improve robustness and reduce overfitting risk. AutoNCP extends this idea by constructing a structured sparse circuit automatically given desired sizes.

2.3.2 Liquid time-constant networks (overview)

LTC networks represent a recurrent computation as a continuous-time dynamical system whose effective time scale adapts to the input, in the same broad spirit as neural ordinary differential equation models that parameterise continuous dynamics [Che+18]. A common way to express the idea is through a state equation where the rate of change of the hidden state depends on both the current state and the current input. In simplified form, it can be described as:

$$\dot{x}(t) = -\frac{1}{\tau(x(t), u(t))} x(t) + f(x(t), u(t)),$$

where $x(t)$ is the hidden state, $u(t)$ is the input, $\tau(\cdot)$ is an input- and state-dependent time constant, and $f(\cdot)$ is a learned function. The exact form differs by implementation, but the key idea remains: the system can respond quickly to some inputs and more slowly to others.

The key conceptual difference from standard discrete-time RNNs is:

- in standard RNNs (including gated variants), the recurrence is applied at a fixed discrete rate with gating controlling information flow,
- in LTC, the internal dynamics behave like a continuous-time system where the response speed can change depending on the signal.

This adaptive time-scale behaviour is appealing for tasks that mix fast and slow signals, and for long-horizon problems where stability of recurrent dynamics matters. In essence, an LTC model is designed to process different input signals at different effective speeds, retaining some information over longer horizons while reacting quickly to short-term

changes. Instead, it can keep some information for longer while still reacting quickly when a short-term change is important.

2.3.3 AutoNCP wiring and neural circuit policies

AutoNCP wiring constructs a sparse recurrent circuit organised into conceptual neuron groups. A common description partitions units into:

- **sensory neurons** (receive inputs),
- **interneurons** (mediate intermediate processing),
- **command neurons** (higher-level aggregation),
- **motor neurons** (drive outputs).

Sparse wiring matters for several practical reasons:

- fewer recurrent parameters and a stronger inductive bias,
- potentially reduced overfitting in rich observation settings,
- improved interpretability relative to dense recurrence (at least structurally).

For CPP, the policy must process both structured spatial inputs and scalar context such as remaining budget and return urgency. A structured recurrent core is therefore an attractive way to maintain a compact memory while upstream branches encode local and global spatial information.

2.3.4 LTC versus LSTM and GRU: conceptual comparison

Readers are often familiar with LSTM [HS97] and GRU [Chu+14]. A useful way to compare them with LTC is along four dimensions:

- **State update mechanism:** LSTM/GRU update hidden state in discrete time with gates; LTC uses continuous-time-inspired dynamics with input-dependent time constants.

- **Temporal processing style:** LSTM/GRU process temporal signals at a fixed update rate; LTC can adapt its effective time scale to the current input regime.
- **Stability properties:** the LTC literature emphasises bounded dynamics and stable behaviour under certain conditions; LSTM/GRU stability is typically addressed empirically via training heuristics and gating.
- **Relevance for CPP:** CPP requires both local, fast reactions (avoid obstacles, choose a legal move) and long-term context tracking (visited areas, remaining budget). LTC is theoretically appealing because different temporal patterns can be processed at different effective speeds.

This comparison is not intended to dismiss LSTM/GRU. The point is to motivate LTC as a principled alternative recurrent mechanism that may be well matched to coverage tasks with mixed time scales and partial observability.

2.3.5 Prior applications of LTC in robotics

LTC and related liquid models have been studied in robotics and control problems such as navigation and closed-loop control [MPG24]. These are tasks where the agent must act under partial observability and where the dynamics can change over time. In many robotics settings, the agent also faces noisy sensors and must make decisions from imperfect information. Recurrent models are a natural fit in such cases because they can integrate information over time.

Liquid models are often discussed as an alternative to standard gated recurrence because of their continuous-time formulation and the idea of input-adaptive dynamics. Prior work in this area supports the feasibility of applying LTC-like recurrence in embodied decision problems.

At the same time, CPP differs from many benchmark control tasks. Coverage is explicitly **history dependent**: the benefit of an action depends on what has already been covered. CPP also often has **hard constraints** such as a movement budget and safety constraints that can force termination or forced return. Finally, the multi-agent case adds a coordination layer that couples the agents' decisions.

This motivates the question addressed by this thesis: whether an LTC-based recurrent policy trained with PPO can serve as the core decision model for 3D multi-agent CPP under realistic constraints.

2.3.6 Why LTC suits CPP: conceptual match

Even before describing the implementation, the CPP task has several properties that map naturally to the LTC/NCP motivation:

- **Trajectory memory:** the policy benefits from remembering which regions were previously covered, which reduces redundant revisits.
- **Multi-timescale processing:** local occupancy and legal actions change step-to-step, while coverage and remaining budget evolve slowly.
- **Generalisation:** adaptive temporal processing can help when the environment is procedurally generated and not repeated exactly.
- **Sparse recurrence:** a structured recurrent core can act as a compact memory when the observation is rich (for example, volumetric grids plus scalar context).

These arguments justify investigating LTC as the recurrent core in a CPP policy, while leaving the exact model structure, feature extraction, and training protocol to Chapter 3.

2.4 Related work and research gap

This section situates the thesis in three partly overlapping strands of literature: (i) coverage path planning and learning-based variants, (ii) multi-agent reinforcement learning for cooperative UAV-style tasks, and (iii) liquid time-constant (LTC) recurrent networks as a distinct class of temporal models. The goal is not to survey every paper in each strand, but to make the *motivation* for the present work explicit and to state clearly what appears to be *novel*: combining an LTC-based policy with PPO for **3D multi-agent CPP** under terrain, no-fly zones, and budgeted safety, a combination that, to the best of my knowledge, has not been published in the CPP literature.

2.4.1 Coverage path planning and reinforcement learning

Classical CPP is a well-studied family of problems: given a region and a sensor footprint, construct a path that achieves complete or near-complete coverage while respecting kinematic or geometric constraints. Exact decomposition, sweep-based methods, and graph-based heuristics remain common when the environment is static, fully known, and low-dimensional. In many real deployments, however, the environment is only partially known, may change between missions, or is easier to model procedurally than to specify manually for every instance. That motivates *learning* policies that can reuse behaviour across episodes and generalise to new layouts.

In recent years, deep reinforcement learning has been applied to coverage-style objectives for UAVs and mobile robots [CBF19; BD22]. These works commonly use map-like observations, action masking, and constraints reflecting energy budgets, varying power, or continuous dynamics. A representative line of work formulates coverage in continuous or large-scale settings and trains deep policies with actor–critic methods; for example, Theile et al. propose continuous-world coverage path planning for fixed-wing UAVs with deep reinforcement learning, using curvature-constrained motion and learned policies in a setting that differs from the present voxel-grid formulation but illustrates the same broad trend: the use of reinforcement learning for CPP as an alternative to purely geometric planners [The+25]. Survey and review articles on UAV coverage path planning similarly emphasise the trade-off between classical optimality guarantees and learning-based adaptivity under uncertainty.

Across this literature, several patterns recur. Most published RL-based CPP studies focus on **single-agent** settings, **2D** or planar abstractions, or constraints that are simpler than the full stack used here (3D terrain occupancy, volumetric no-fly zones, *and* a hard per-agent movement budget coupled to return-to-safe-zone logic). Architecturally, policies are typically built from convolutional or feed-forward backbones, sometimes with conventional RNNs (LSTM/GRU) for memory, but not with liquid time-constant recurrent cores. The present thesis therefore does not claim to replace the entire CPP literature; it claims a specific gap at the intersection of **constraint-rich 3D CPP** and **liquid recurrent policies**.

2.4.2 Multi-agent reinforcement learning for coverage and navigation

Multi-agent reinforcement learning (MARL) provides a framework for learning decentralised or partially decentralised policies when several agents share the same environment. Centralised training with decentralised execution (CTDE) is a standard recipe when global information is available at training time but not at deployment time, and multi-agent actor-critic methods [Low+17] exemplify how to learn joint value functions while producing policies that each agent can run independently.

In coverage and UAV navigation, MARL has been used to learn coordination, reduce redundant overlap, and handle interaction between agents [Tam+17]. However, published work in this space typically assumes **standard neural backbones** (MLP, CNN, Transformer-style attention over tokens, or conventional RNNs) and does not investigate LTC as the recurrent core in a CPP-specific policy. Moreover, many multi-agent coverage experiments simplify geometry (e.g., 2D grids or coarse maps) or omit the full combination of **3D terrain**, **restricted airspace**, and **budgeted forced-return** that the environment in this thesis encodes. That gap matters for the motivation of this work: coordination is necessary but not sufficient; the question is whether a **liquid recurrent policy** can learn both coverage and cooperation under those constraints.

2.4.3 Liquid time-constant networks and prior robotics use

Liquid time-constant networks (LTCs) were introduced as a class of continuous-time-inspired recurrent models in which dynamics are built from interlinked first-order systems with input-dependent time constants, yielding “liquid” temporal behaviour [Has+21]. Theoretical work also studies LTCs as universal approximators in the recurrent setting [Has+18]. These papers establish LTC as a *distinct* alternative to LSTM/GRU for temporal modelling, with emphasis on stability and expressivity in trajectory space.

Subsequent applications in robotics and control (navigation, tracking, closed-loop control) show that LTC-style recurrence can be useful when observations are noisy and when multiple time scales coexist. However, **coverage path planning** is not a generic control problem: it is explicitly *history-dependent* (every cell can be visited at most once for “first coverage” credit), and the reward structure couples long horizons to local legality. Prior LTC robotics work does not address CPP as a task family, nor does it evaluate LTC inside a **3D multi-agent CPP** simulator with NFZ and terrain.

2.4.4 Motivation for combining PPO, LTC, and multi-agent CPP

Proximal policy optimisation (PPO) [Sch+17] is a practical on-policy algorithm for training large policies with stable updates; it is widely used when rollouts are expensive and policy stability matters [Hen+18]. For CPP with long episodes and recurrent policies, PPO is a natural training choice alongside truncated backpropagation through time (TBPTT) for the recurrent core [Wer90].

The **motivation** for placing an LTC at the centre of the policy is threefold. First, CPP requires remembering what has already been covered and how much budget remains; a recurrent core with adaptive time scales is a plausible match for mixed fast (local collision avoidance) and slow (coverage progress) signals. Second, procedural generation of terrain and NFZ means that the policy must generalise across layouts; recurrent representations that adapt their effective dynamics to the input regime may help compared to a purely feed-forward CNN. Third, from a research perspective, **LTC has been studied in robotics and in time-series modelling, but not in the CPP literature as the main policy backbone**, so the question is not only whether “some” network can solve CPP, but whether this *family* of networks is worth adopting in this domain.

2.4.5 Research gap and novelty of this thesis

Prior work addresses pieces of the puzzle in isolation. Classical CPP planners offer coverage guarantees in stylised settings. RL-based CPP and UAV coverage papers show that deep policies can learn coverage under constraints, but typically with conventional architectures and often in 2D or single-agent settings [The+25]. MARL provides coordination tools [Low+17] but does not, in the surveyed literature, combine LTC with CPP. The LTC literature [Has+21; Has+18] motivates liquid recurrence for robotics and time series, but does not evaluate CPP.

The **research gap** addressed here can be stated plainly: **To the best of my knowledge, there is no published work that applies Liquid Neural Networks (including LTC) to coverage path planning as an end-to-end learned policy for area coverage.** In other words, the two lines of research, CPP and LiquidNN/LTC have not been merged in the literature in the way this thesis proposes.

2 Background

The **novelty** of the work is therefore not a single hyperparameter but the *combination*: (i) a **3D multi-agent** CPP formulation with terrain, volumetric no-fly zones, and hard movement budgets with forced-return safety; (ii) an **LTC-based recurrent policy** with attention-based inter-agent communication, trained with **PPO** [Sch+17]; and (iii) an empirical evaluation that ties training dynamics, checkpoint selection, and fixed-seed evaluation to the stated research questions. This thesis is intended as a first step in that direction: establishing feasibility and reporting behaviour under a controlled experimental protocol.

3 Methods

3.1 Environment design

3.1.1 3D grid world

I model the task as a discrete 3D voxel grid world. The grid size is $20 \times 20 \times 6$ in x , y , and z . Each episode is generated procedurally using a random seed so that the learned policy must generalise across layouts rather than memorise a fixed map.

Bialas and Döller [BD22] study coverage path planning for UAVs in complex 3D environments with deep reinforcement learning, demonstrating that learned policies can cope with non-trivial three-dimensional structure and obstacles. The environment in this thesis is aligned with that goal, 3D coverage under constraints but instantiates it as a *voxel* grid with six-connected moves, procedural terrain and volumetric no-fly zones, two agents, and hard movement budgets with forced return (Sections 3.1.2–3.1.3). The discrete abstraction trades some kinematic detail for explicit feasibility checks, dense multi-channel observations, and reproducible episode generation.

Terrain generation. Terrain is generated as a 2D height field using a sum of Gaussian peaks and is then discretised to integer heights. Cells at or below the terrain height are treated as solid terrain and are not flyable. An agent state (x, y, z) is flyable only if z is strictly above the terrain height at (x, y) .

No-fly zones (NFZ). Each episode contains a randomly generated NFZ volume. The NFZ is created by sampling a random footprint shape (rectangle, square, circle, ellipse, or triangle) in the horizontal plane and extending it vertically from the local terrain height upward. Any voxel inside the NFZ is blocked for flight. If the NFZ overlaps the

target area excessively, I cap the amount of target area removed by unblocking some NFZ voxels inside the target area to avoid trivialising the task.

Target area (TA). I define the target area as the set of surface voxels one cell above the terrain. Concretely, for each (x, y) column I mark a single target voxel at

$$z = \min(D - 1, h(x, y) + 1).$$

I also compute a 2D TA footprint, that is, which (x, y) columns contain any target voxel, and use it for inside-TA checks and TA-distance features so that agents flying above the TA are still treated as on-task.

Safe landing zones (SLZ). A set of SLZ voxels is created by selecting feasible landing cells on the TA surface layer, just above terrain, that are not blocked by NFZ. The environment ensures at least $2\times$ the number of agents, with a minimum of 4, SLZ positions when possible. SLZ positions serve both as initial spawn points and as safe return destinations under the budget constraint.

Agent spawning. I use two agents. At reset, I select spawn SLZ positions that are far apart in 2D Manhattan distance to reduce immediate interference. Here, Manhattan distance between two grid cells (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2| + |y_1 - y_2|$, which matches the idea of moving on a grid with axis-aligned steps. If insufficient distant combinations exist, I use a greedy farthest-point selection as a fallback. Agents spawn on SLZ voxels and immediately mark their starting locations as visited and covered.

Coverage and visibility. Coverage is tracked as a 3D boolean grid of visited and covered voxels. In addition to marking the agent’s current cell, I use a visibility-based mechanism that casts directional cones along $\pm x$, $\pm y$, and $\pm z$. Voxels in the cone are marked as covered if line of sight is not blocked by terrain, NFZ, or other agents. This creates a sensing model in which movement reveals nearby space and provides a dense signal for learning.

Observation representation. The observation is a flat vector that consists of (i) a 7-channel grid, (ii) per-agent scalars, and (iii) per-agent legal action masks.

The 7 grid channels, in fixed order, are as follows:

- **Ch0 covered:** whether each voxel has been covered.
- **Ch1 NFZ:** whether each voxel is inside a no-fly zone.
- **Ch2 agent 0 position:** one-hot voxel occupancy of agent 0, if not landed.
- **Ch3 agent 1 position:** one-hot voxel occupancy of agent 1, if not landed.
- **Ch4 target area:** target-area voxels.
- **Ch5 uncovered target area:** TA voxels that are not yet covered.
- **Ch6 terrain:** solid terrain voxels.

For each agent, I include 11 scalar features. Surveys and learning-based UAV coverage studies argue that policies should see not only spatial structure (maps or grids) but also compact variables for pose, height relative to terrain or obstacles, remaining mission resources, and cues that support safe return or energy-aware behaviour [CBF19; The+20]. Deep RL path planning for UAVs similarly combines global and local map information with goal-relative signals so that a policy can orient coverage without inferring everything from occupancy alone [The+21]. The list below follows that rationale in the discrete voxel setting:

- normalised x , y , and z position,
- terrain-relative altitude fraction,
- normalised return distance to the nearest SLZ based on shortest-path distance,
- remaining budget fraction,
- goal displacement (dx, dy, dz) ,
- inside-TA flag based on the 2D TA footprint,
- normalised 2D distance to the TA footprint based on Manhattan distance in (x, y) .

3 Methods

The terrain-relative altitude fraction is defined as

$$\frac{z - h(x, y)}{D},$$

clipped to $[0, 1]$. This feature makes the observation terrain-aware: the same absolute altitude may be safe in a valley but unsafe above a ridge, and this feature captures that relationship directly.

Finally, I append the legal action mask for each agent, with 6 values per agent, so the policy can suppress invalid moves.

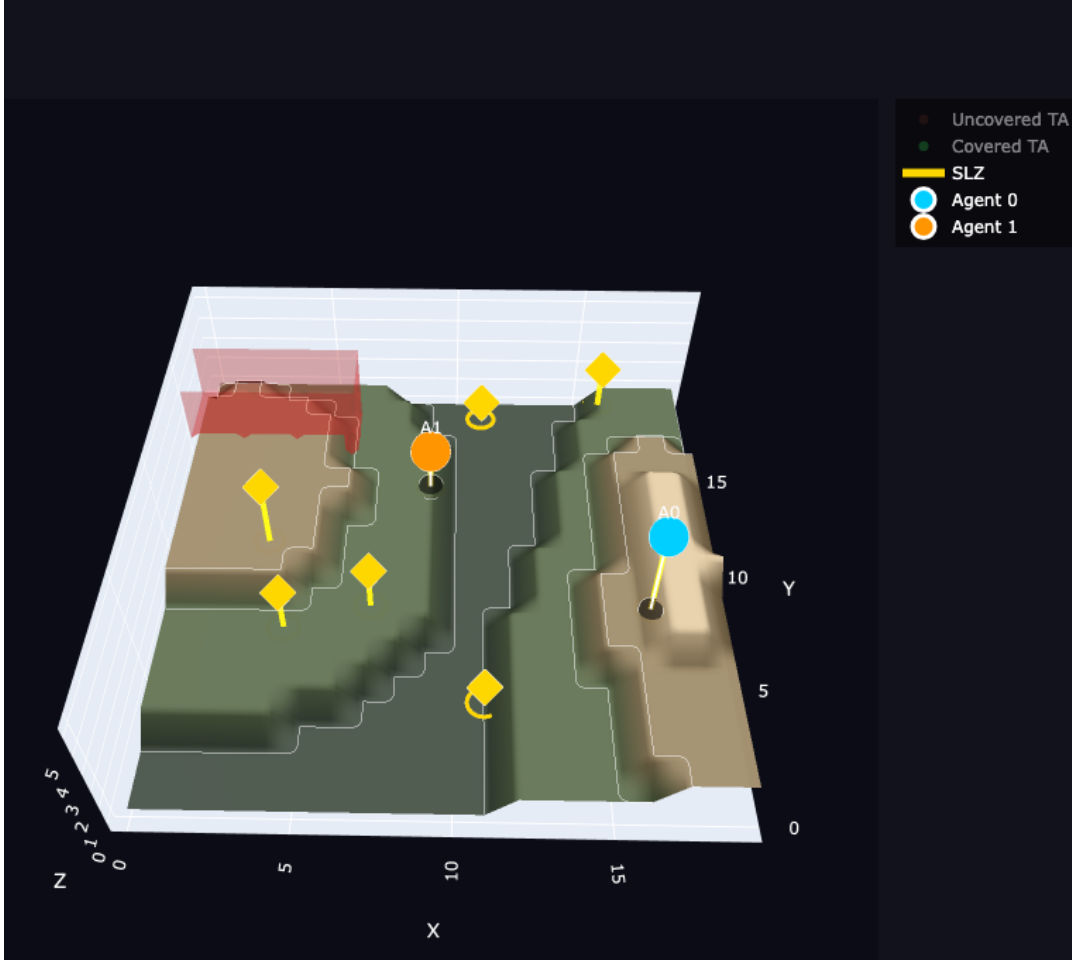


Figure 3.1: Environment view (3D). Terrain, the target-area surface layer, NFZ volumes, SLZ locations, and both agents are shown together.

Figures 3.1–3.3 are included to clarify the task setting before introducing the action space and safety rules. They show how terrain and NFZs restrict flyable space, how SLZs

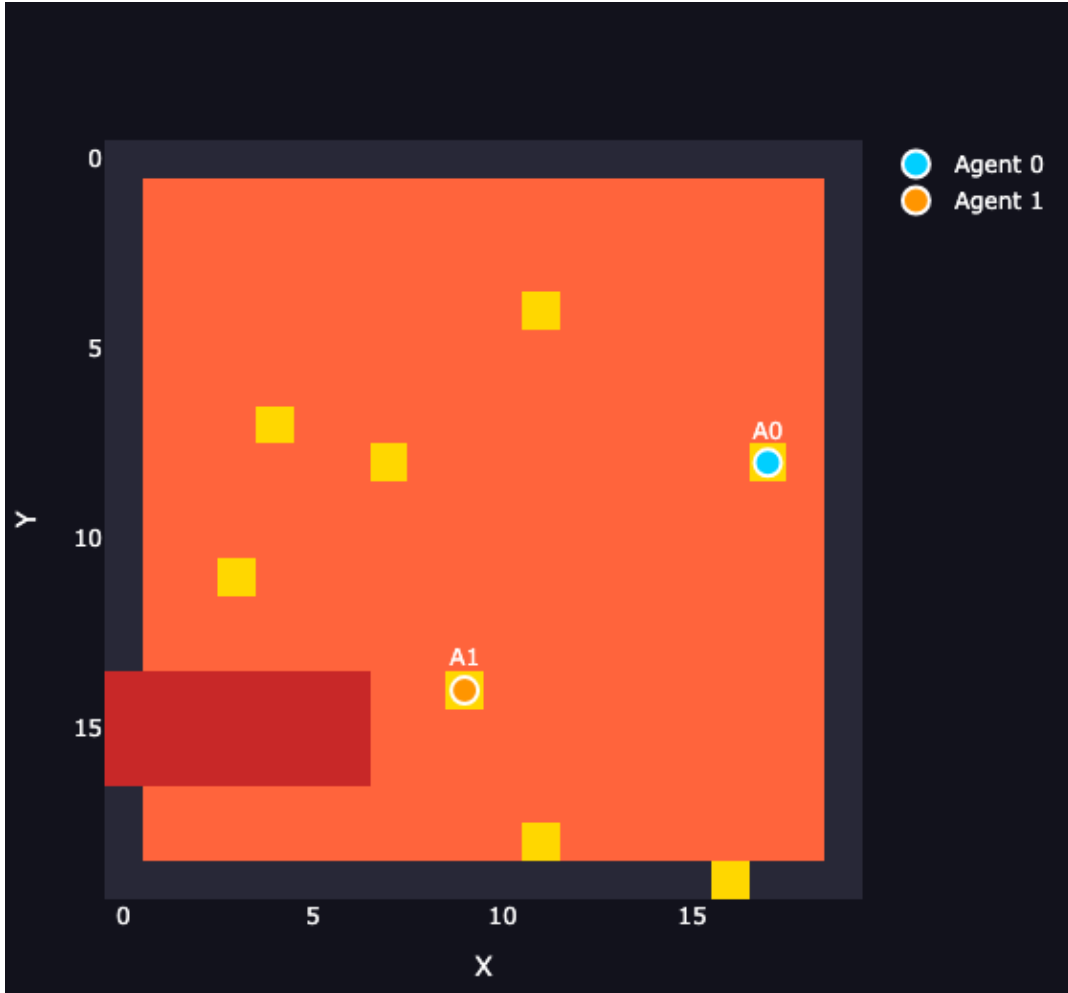


Figure 3.2: Environment view (top-down). This view highlights the horizontal footprint of the TA and NFZ and the spatial relationship between SLZ spawns and the agents.

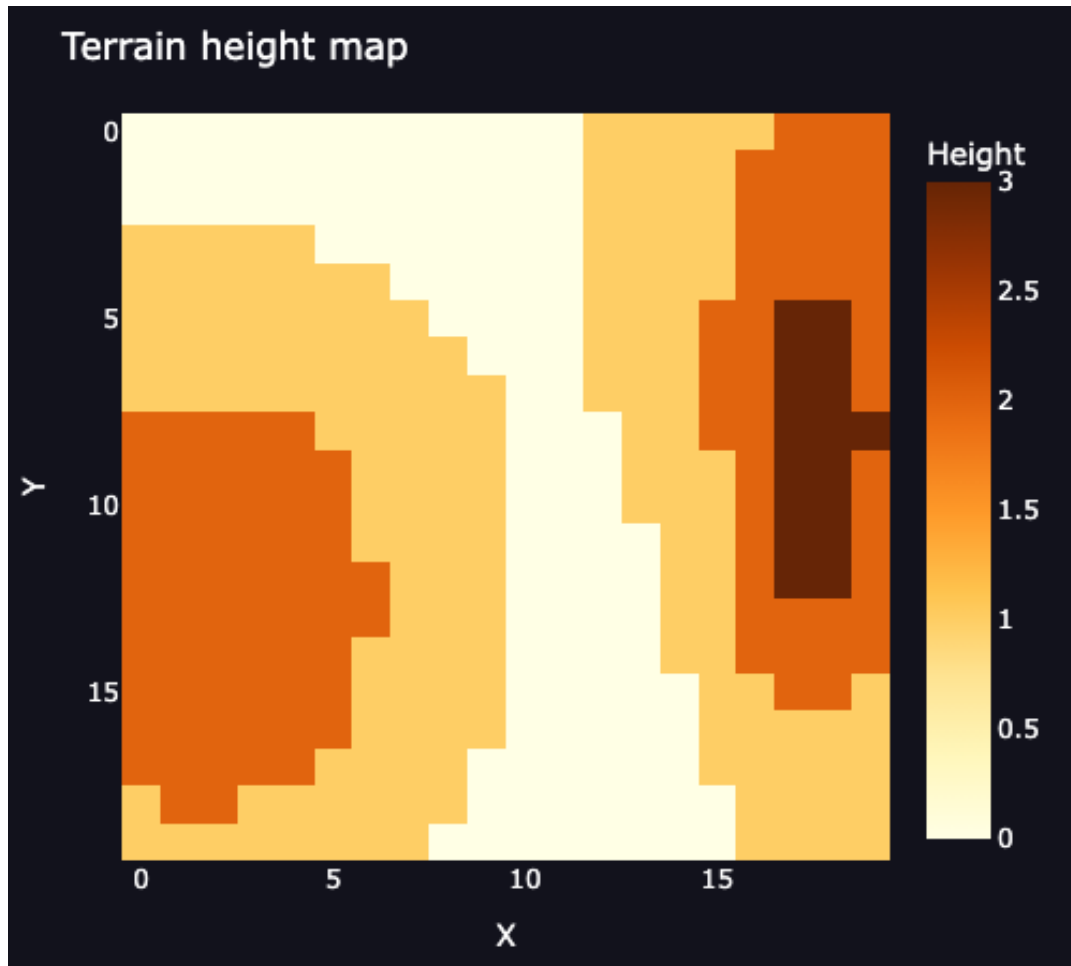


Figure 3.3: Environment view (terrain heatmap). Peaks and valleys explain why the same absolute altitude can be safe in one column but infeasible in another.

provide safe start/return locations, and why coverage is naturally a 3D, history-dependent objective in this environment.

3.1.2 Action space, masking, and episode termination

Each agent has a discrete 6-action movement space:

- $+x$,
- $-x$,
- $+y$,
- $-y$,
- $+z$,
- $-z$.

Each action corresponds to a **unit move by one voxel** along a single axis in the 3D grid. In other words, the agent moves to one of its **6-connected neighbours**: north/south/east/west in the horizontal plane ($\pm x, \pm y$) or one step up/down in altitude ($\pm z$). I use this minimal action set to keep the dynamics simple and to make the effect of constraints (terrain, NFZ, boundaries, and return feasibility) explicit.

At each step, the environment constructs a legal action mask per agent by checking bounds and flyability, namely, whether the next state is outside terrain, outside NFZ, and strictly above terrain height. The policy outputs logits for all 6 actions, and invalid actions are suppressed before sampling by adding a large negative value to masked logits.

Action masking. I use action masking at every timestep during both rollout collection and evaluation. After computing which of the 6 moves would lead to an invalid next state, out of bounds, inside NFZ, or at or below the terrain surface, I build a binary mask with 1 for legal actions and 0 for illegal actions. This mask is provided to the policy and applied directly to the action logits by adding a very large negative value to illegal actions before the softmax and sampling step. This ensures that the agent never selects physically invalid moves, improves learning stability by avoiding wasted probability mass on impossible actions, and makes exploration focus on meaningful transitions.

An episode terminates when any of the following conditions is met:

- **full coverage achieved:** TA coverage is at least 0.99,
- **all agents landed:** agents return and land on SLZ under forced-return mode,
- **budget exhausted / no one can move:** budgets are depleted such that no further action is possible.

3.1.3 Budget constraint

Each agent has a hard movement budget of 200.0. A valid movement step consumes a fixed cost of 1.0. Safety is enforced by requiring that an agent must be able to return to an SLZ before budget exhaustion.

To implement this, at each step I compute an approximate required return budget by finding the shortest 6-connected path length from the agent’s position to the nearest SLZ while treating terrain, NFZ, and other active agents as obstacles. If the remaining budget falls below the required return budget plus a safety buffer of 5 steps, the agent enters *forced-return mode*.

In forced-return mode, the action executed for that agent is taken from the precomputed shortest path back to an SLZ. This mechanism prevents catastrophic budget depletion away from safe landing locations and allows me to report diagnostics about when forced return was initiated.

This design is directly connected to the logged diagnostics:

- **forced return step (per agent):** the timestep when forced return was initiated,
- **forced return coverage (per agent):** TA coverage at the time forced return was initiated.

3.1.4 Episode seeding and environment diversity

The environment is randomised across episodes via procedural generation, including terrain, NFZ shape and placement, SLZ positions, and spawn selection. Episode layouts are drawn using a fixed reproducible seeding schedule (Subsection 3.4.1) so that the same training code always produces the same sequence of environments for a given master seed. The results chapter reports learning curves and summaries over that complete run and evaluates saved checkpoints on fixed evaluation seeds.

3.2 Reward function

3.2.1 Reward design philosophy

The reward function is the primary mechanism that encodes the desired behaviour. The design balances dense encouragement for coverage and exploration against penalties for inefficient movement patterns, unsafe interactions, and time spent outside the target area.

3.2.2 Individual reward components

For each component below, I describe what it is, how it is computed, what behaviour it encourages or discourages, and its empirical scale in the logged training run.

Reward visibility (reward_visibility).

- **What:** reward for newly covered target voxels revealed by the visibility model.
- **How computed:** after each movement, I cast visibility cones in 6 directions and count the number of newly covered TA voxels; the reward is proportional to this count.
- **Behaviour:** encourages movement that reveals new parts of the TA and discourages stagnation.
- **Observed scale:** mean 212.742655, min 20.0, max 307.000000

First visit (`reward_first_visit`).

- **What:** bonus for first-time visits to new cells inside the TA footprint.
- **How computed:** if the agent moves to a cell with zero prior visits and the (x, y) column is inside the TA footprint, I add a fixed bonus of 0.25 per first visit. This is based on the footprint rather than the exact TA surface voxel, so the agent can receive the bonus while flying above the TA column as well as when visiting the TA surface cell.
- **Behaviour:** encourages first-time exploration in the TA region and reduces wasted movement in free air outside the TA. In combination with the outside-TA penalty, this term helps keep behaviour on-task by making new progress inside the TA footprint consistently more rewarding than drifting outside.
- **Observed scale:** mean 34.763641, min 1.5, max 67.5.

Revisit penalty (`reward_revisit_penalty`).

- **What:** penalty for revisiting previously visited cells.
- **How computed:** if a moved-to cell has a non-zero prior visit count, I subtract a fixed penalty of 0.05 per revisit event.
- **Behaviour:** discourages oscillation and redundant movement.
- **Observed scale:** mean -9.022416 , min -17.35 , max -0.200000 .

Collision penalty (`reward_collision`).

- **What:** penalty for agent-agent collisions or swap conflicts.
- **How computed:** if multiple agents propose the same next voxel, or attempt a swap into each other's current positions, I apply a collision penalty of 5.0 per collision event; a swap applies twice.
- **Behaviour:** discourages unsafe overlap and direct interference.
- **Observed scale:** mean -0.63 , min -85.00 , max 0.0 .

Full coverage reward (reward_full_coverage).

- **What:** terminal bonus for achieving full coverage.
- **How computed:** if final TA coverage reaches at least 0.99, I add a terminal reward of 30.0 and terminate the episode.
- **Behaviour:** provides a strong end-condition incentive for completing coverage.
- **Observed scale:** mean 3.88, min 0.0, max 30.0.

Coverage milestone bonus (reward_coverage_bonus).

- **What:** bonuses when coverage crosses predefined milestones.
- **How computed:** when coverage first exceeds each milestone in $[0.70, 0.75, 0.80, 0.85, 0.90, 0.95]$, I add a scaled bonus proportional to the milestone level.
- **Behaviour:** shapes the learning signal to reward steady progress rather than only final coverage.
- **Observed scale:** mean 277.87, min 0.0, max 521.05.

Outside-TA penalty (reward_outside_ta).

- **What:** penalty for being off-task outside the TA footprint.
- **How computed:** for each non-landed agent, if its (x, y) position is outside the 2D TA footprint, I subtract a fixed penalty of 0.10 per step.
- **Behaviour:** encourages agents to remain on-task and return to the TA region.
- **Observed scale:** mean -14.75 , min -38.40 , max 0.0.

Overall, the dominant positive terms are the milestone bonus and the visibility-based coverage reward. Penalties mainly shape efficiency through the revisit penalty and safety or task focus through collision and outside-TA penalties.

Table 3.1: Reward component statistics (logged training run) and qualitative purpose.

Reward component	Mean	Min	Max	Purpose (behaviour shaped)
<code>reward_visibility</code>	212.74	20.00	307.00	Dense reward for newly revealed and covered TA via visibility
<code>reward_first_visit</code>	34.76	1.50	67.50	Encourages first-time exploration inside TA footprint
<code>reward_revisit_penalty</code>	-9.02	-17.35	-0.20	Discourages oscillation and redundant revisits
<code>reward_collision</code>	-0.63	-85.00	0.00	Discourages agent-agent conflicts, including same-cell and swap conflicts
<code>reward_full_coverage</code>	3.88	0.00	30.00	Terminal incentive to complete coverage
<code>reward_coverage_bonus</code>	277.87	0.00	521.05	Shapes progress across coverage milestones
<code>reward_outside_ta</code>	-14.75	-38.40	0.00	Keeps agents on-task near or over the TA footprint

Interpreting Table 3.1. Mean, min, and max values are computed over all logged episodes in the training run. These statistics summarise the episode-level aggregates of each reward component recorded from the environment. Positive components add to the episode-level `total_reward`, whereas negative components correspond to penalties (e.g., revisit and collision costs). Notably, `reward_full_coverage` is a terminal bonus assigned only when final target-area coverage reaches at least 0.99 and the episode terminates with full coverage; it is zero otherwise. Consequently, the coverage curves may indicate frequent near-complete progress, while `reward_full_coverage` remains small in expectation because it reflects only the subset of episodes that satisfy the strict terminal coverage threshold.

3.3 Policy architecture: LNN policy

The policy must jointly process spatial structure in 3D grids, temporal history (coverage is path-dependent), per-agent scalar features such as budget and return urgency, and inter-agent information for coordination and collision avoidance. I address this using a multi-branch feature extractor, an LTC recurrent core, and attention-based agent communication.

3.3.1 Feature extraction pipeline

At each timestep, I construct features from the 7-channel 3D grid and the per-agent scalar features.

Global branch. I compress global grid information using summary statistics. I select 5 global channels, covered, NFZ, target, uncovered target, and terrain, and compute both the mean and max over all voxels. The resulting global statistics vector is passed through an MLP to produce a compact global feature representation.

Local 3D branch. For each agent, I extract a 3D crop centred at the agent position with size $(2h + 1)^3$, where h is the crop half-width (in the experiments reported here, $h = 2$). I process this crop with two Conv3D layers and adaptive average pooling to produce a fixed-size local volumetric feature.

Local 2D branch. From the same 3D crop, I extract the horizontal slice at the agent’s current altitude and project it through an MLP. This provides a lighter-weight representation of local planar structure at the current flight level.

Scalar branch. I process two urgency scalars per agent: normalised return distance and remaining budget fraction. This branch encodes return urgency independently of the spatial features.

Goal branch. I process the goal displacement (dx, dy, dz) , which points either toward the nearest uncovered target voxel, when already over the TA footprint, or toward the centroid of the TA footprint, when outside the footprint. This provides directional guidance that is stable under terrain and NFZ obstacles.

Uncovered-direction branch. I compute a coarse directional summary of uncovered target mass within the local crop by summing uncovered target voxels in the $+x$, $-x$, $+y$, $-y$, $+z$, and $-z$ halves of the crop. This 6D vector is passed through an MLP to produce a directional feature that encourages movement toward locally uncovered regions.

All per-agent features are concatenated, and features from all agents are assembled into a single per-timestep feature vector that is used as input to the recurrent core.

3.3.2 Liquid Time-Constant (LTC) recurrent core

I use a Liquid Time-Constant (LTC) recurrent core to process the per-timestep feature sequence [Has+21; Lec+19]. Compared to a feedforward model, the recurrent core provides memory of past visited regions and progress. Compared to a standard recurrent model with fixed-rate recurrence, the LTC provides continuous-time, input-adaptive temporal dynamics.

Implementation details are as follows:

- the recurrent core is `ncps.torch.LTC` using an `AutoNCP` wiring,
- the per-episode hidden state is carried across timesteps and reset only at episode boundaries,
- during rollout collection, I initialise the hidden state to zeros for a new episode and pass it forward step by step with the policy,
- during PPO updates, I train through time using truncated backpropagation through time (TBPTT) [Wer90] by splitting rollouts into fixed-length chunks, with chunk size 100, and seeding each chunk with the stored hidden state from rollout collection.

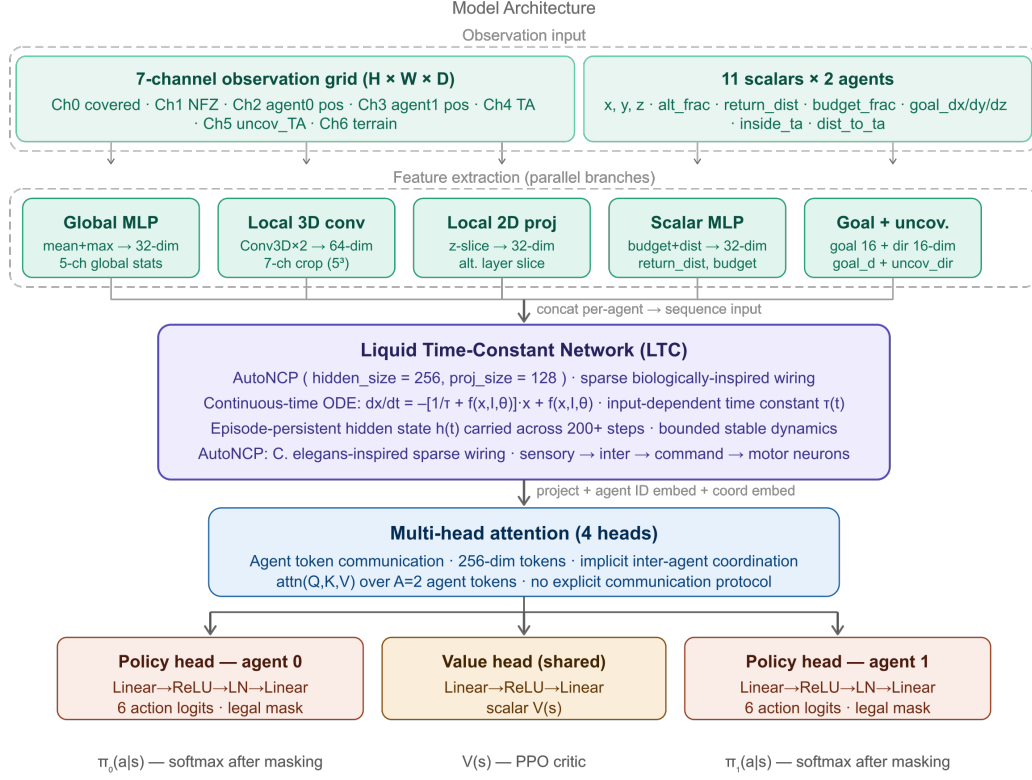


Figure 3.4: Policy architecture overview. The observation is processed by parallel feature branches (global statistics, local 3D and 2D views, scalar urgency, and goal/uncovered-direction features). The concatenated per-timestep features are passed through an LTC recurrent core (AutoNCP wiring) and then through multi-head attention for inter-agent communication. The network outputs one masked 6-action policy head per agent and a shared value head for PPO.

This design allows the policy to integrate both fast-changing signals, such as local occupancy and near-term movements, and slow-changing signals, such as accumulated coverage and remaining budget, over long horizons.

Table 3.2: Dimensions for feature branches and heads.

Component	Input	Notes
Global branch (stats MLP)	$5 \text{ channels} \times \{\text{mean}, \text{max}\} = 10$	Output dimension: 32. Statistics are computed over the full $H \times W \times D$ grid.
Local 3D branch (Conv3D + pool)	$7 \times (2h+1)^3$	Output dimension: 64. Uses the full 7-channel local crop; default $h = 2$.
Local 2D branch (slice MLP)	$7 \times (2h+1)^2$	Output dimension: 32. Uses the centred planar slice from the local crop.
Scalar branch (urgency)	2	Output dimension: 32. Encodes normalised return distance and remaining budget fraction.
Goal branch (dx, dy, dz)	3	Output dimension: 16. Directional goal proxy.
Uncovered-direction branch	6	Output dimension: 16. Half-crop sums for $\pm x$, $\pm y$, and $\pm z$.
Per-agent concatenation	Sum of branch outputs	Output dimension: 192. Concatenated per-agent feature before the recurrent core.
LTC core (<code>ncps.torch.LTC</code>)	$2 \times 192 = 384$ per timestep	AutoNCP configured with <code>hidden_size=256</code> and <code>proj_size=128</code> .
Communication token projection	$128 + 64 + 64 = 256$	Combines LTC output (128) with identity and coordinate embeddings (64 each).
Multi-head self-attention (4 heads)	$2 \text{ tokens} \times 256$	Output remains $2 \text{ tokens} \times 256$. Shown here for the two-agent setting.
Policy head (per agent)	256	Output: 6 masked action logits.
Value head (shared)	Pooled 256	Output: 1 scalar value for PPO.

3.3.3 Attention-based agent communication

After the LTC produces a temporal feature representation at each timestep, I implement inter-agent communication using multi-head self-attention [Vas+17].

For each agent, I construct a communication token by combining:

- the shared LTC output feature, broadcast to agents,
- an agent identity embedding,
- a coordinate embedding derived from the 11 scalar features.

These tokens are projected and passed through multi-head attention with 4 heads. The attention output provides each agent with a representation conditioned on the other agent, enabling implicit coordination without an explicit centralised planner.

3.3.4 Policy and value heads

I use per-agent policy heads and a shared value head:

- **Policy head (per agent):** a small MLP produces action logits for the 6 discrete moves. I apply legal action masking to suppress invalid actions before sampling.
- **Value head (shared):** I average the attention outputs across agents and predict a single scalar value estimate for PPO.

3.4 Training and evaluation protocol

3.4.1 Training protocol

This section defines the training protocol, including environment generation, reward design, policy architecture, PPO settings [Sch+17; Sch+18], and rollout seeding. Checkpoints are saved during training and evaluated on the fixed-seed suites described below.

Each PPO update gathers rollouts from twelve episodes. Procedural layout generation is relatively expensive, so the same drawn terrain, NFZ, and target-area configuration

is kept for three consecutive episodes after each fresh reset; the twelve episodes in an update therefore use four distinct layouts ($12/3 = 4$). An episode is limited to at most 1000 environment steps and may end sooner when coverage succeeds or budgets are exhausted.

The environment emits an unscaled scalar step reward (sum of the shaped terms in Section 3.2). For optimisation only, each PPO update standardises the collected rewards for that batch by subtracting their mean and dividing by their standard deviation; the stored value estimates used in generalised advantage estimation are shifted and scaled with the same factors so that returns and advantages remain consistent. After computing advantages, they are additionally normalised to zero mean and unit variance within the batch before the clipped surrogate update. Logged episode totals and CSV breakdowns remain on the original reward scale.

Training is reproducible: environment randomness follows a fixed arithmetic rule indexed by the PPO update index $u = 0, 1, 2, \dots$. Let m denote the master seed fixed for the whole training run. For update u , the base seed is

$$s_{\text{base}}(u) = m + 997u,$$

and successive layout identifiers are separated by a stride $\Delta s = 479$. The four layouts used in that update receive seeds $s_{\text{base}}(u) + k\Delta s$ for $k \in \{0, 1, 2, 3\}$. The three episodes that reuse a layout share its seed and hence the same map; policy sampling remains stochastic, so trajectories still differ across those episodes.

Table 3.3: Final hyperparameters and architecture settings used in the reported experiments.

Group	Parameter	Value
Environment	Grid size (H, W, D)	$(20, 20, 6)$
Environment	Local crop half-width h	2
Environment	Movement budget per agent	200.0
Environment	Return buffer / termination	5 steps buffer for forced return when unsafe; episode ends on full coverage, all agents having landed or budget exhaustion
Environment	Number of terrain peaks	3
Environment	NFZ enabled and shape distribution	Enabled; shape $\sim$ uniform over {rectangle, square, circle, ellipse, triangle}
PPO	Learning rate	Warmup 10 updates: $5 \times 10^{-5} \rightarrow 1 \times 10^{-4}$, then 1×10^{-4}
PPO	Discount γ	0.99
PPO	GAE λ	0.95
PPO	Clip ϵ	0.2
PPO	Entropy coefficient	0.05 annealed to 0.005 (episodes 400–1500)
PPO	Value coefficient	0.5
PPO	PPO epochs per update	6
PPO	Maximum gradient norm	0.5
Rollout / data collection	Episodes per PPO update	12
Rollout / data collection	Consecutive episodes per layout draw	3 (same layout, fresh reset)
Rollout / data collection	Distinct layouts per update	4
Rollout / data collection	Maximum steps per episode	1000
Rollout / data collection	Evaluation seeds	100 fixed seeds (held out from training)
Rollout / data collection	Layout seed schedule	$s_{\text{base}}(u) = m + 997u$; stride $\Delta s = 479$; $k \in \{0, 1, 2, 3\}$
Recurrent training	TBPTT chunk size	100
Policy / model	LTC (AutoNCP) 40	Recurrent state size 256; projection size 128
Policy / model	Attention heads	4

3.4.2 Four-checkpoint evaluation strategy

Evaluating a single checkpoint is not sufficient in stochastic reinforcement learning because one episode, or a short window of episodes, can misrepresent quality when layouts and sampling noise vary. I therefore retain models under four selection rules; the results and discussion chapters refer to these saves by short tags so they can be matched to the evaluation tables.

- **Best single episode** (`eval_best`): checkpoint associated with the highest return observed in any individual training episode.
- **Best 100-episode rolling mean** (`eval_last_100`): checkpoint whose mean return over the preceding 100 episodes is largest when the checkpoint is written.
- **Best 500-episode rolling mean** (`eval_last_500`): checkpoint whose mean return over the preceding 500 episodes is largest when the checkpoint is written.
- **Latest training state** (`eval_latest`): most recent parameters before training stopped.

These rules emphasise different notions of performance. The best-episode rule can privilege a rare spike; the rolling-mean rules summarise recent behaviour over different horizons; the latest snapshot reflects where optimisation ended, which need not coincide with the best rolling average.

For fair comparison, I evaluate all four checkpoints on the same 100 fixed seeds, distinct from training seeds, under two movement-budget settings, 100 and 150. This allows a controlled comparison of checkpoint quality and generalisation across different budget regimes.

4 Results

4.1 Training dynamics

Training ran for 11,436 episodes using the protocol described in Chapter 3. This chapter reports the full training log, using the complete episode index throughout.

4.1.1 Coverage trend and end-of-training status

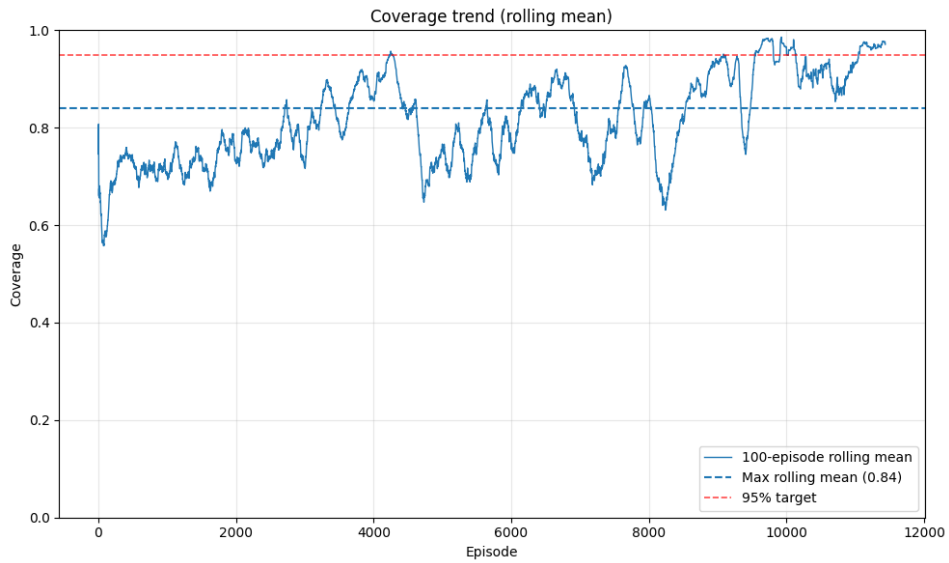


Figure 4.1: Coverage trend (100-episode rolling mean).

The rolling mean rises from roughly the mid-0.5 range to near 1.0; later in training it spends most episodes at or above the 95% reference line, with short-term oscillations.

4 Results

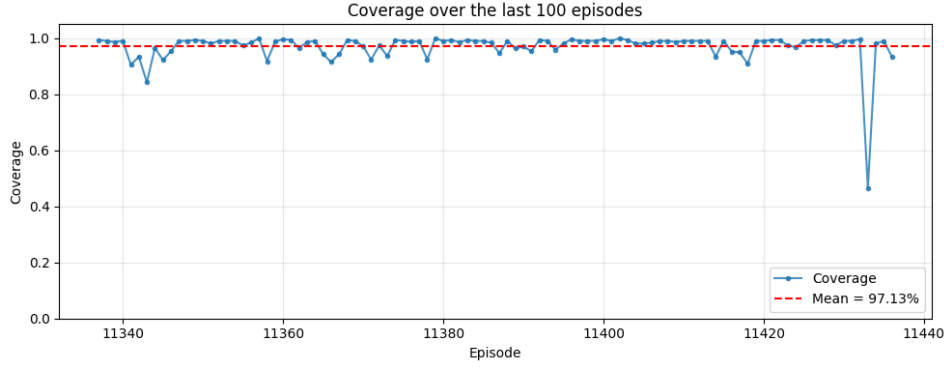


Figure 4.2: Coverage over the last 100 episodes.

Late training is saturated (plot mean 97.13%), with rare low-coverage dips.

Table 4.1: Trailing-window coverage summary (same run).

Trailing window	Mean coverage	Max coverage	Std
Last 50 episodes	0.9704	1.000	0.0754
Last 100 episodes	0.9713	1.000	0.0577
Last 200 episodes	0.9705	1.000	0.0654
Last 500 episodes	0.9633	1.000	0.0798

Means stay above $\sim 96\%$; the last-100 window has the tightest spread (std 0.0577), consistent with Figure 4.2.

Table 4.2: Coverage by stage (episode index split into early/mid/late thirds of the full run).

Stage	Mean	Std	Min	Max
Early (1–33%)	0.7590	0.1518	0.1905	1.000
Mid (33–66%)	0.8103	0.1602	0.1807	1.000
Late (66–100%)	0.8884	0.1428	0.1918	1.000

4.1.2 Distributional views of training coverage

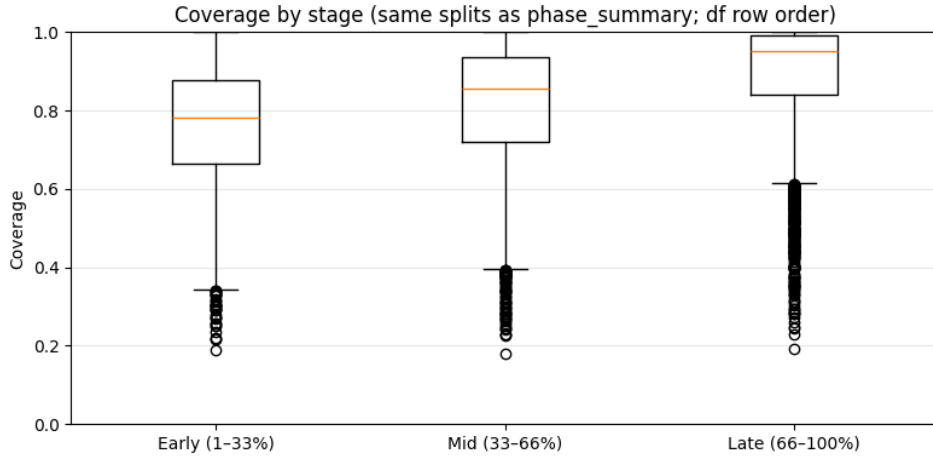


Figure 4.3: Coverage by stage (boxplots; thirds of the full run).

Figure 4.3 summarises the training run by splitting the episode index into three equal thirds (early, mid, late) and plotting the distribution of final episode coverage in each segment. The median and interquartile range shift upward over time, indicating that high-coverage behaviour becomes more typical later in training, while the remaining whiskers/outliers show that occasional low-coverage episodes still occur.

Mean coverage increases by stage; the minimum stays near ~ 0.19 in all thirds, so rare low-coverage episodes persist even late in training.

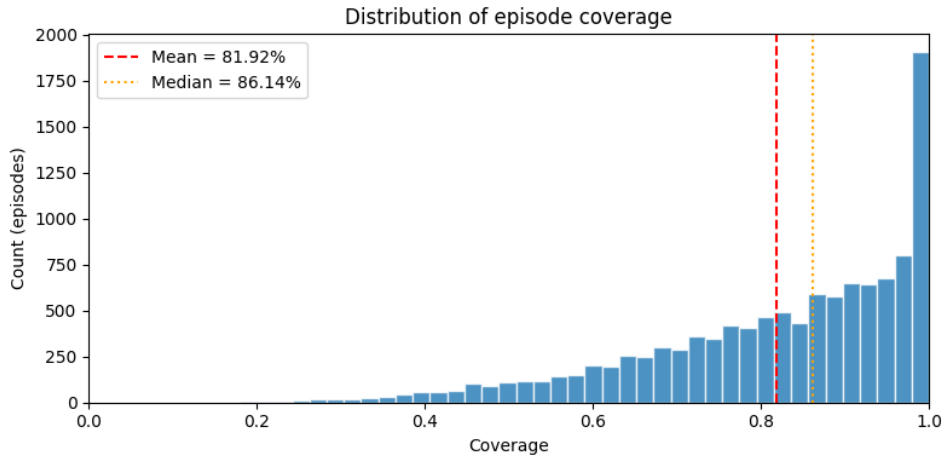


Figure 4.4: Distribution of episode coverage (histogram).

Figure 4.4 aggregates all training episodes into a single histogram of final coverage. The distribution is strongly concentrated near high coverage, which is consistent with a policy that frequently completes most of the target area, while the long left tail represents failure cases (e.g., poor early exploration, getting constrained by NFZ/terrain geometry, or entering forced-return early due to the safety constraint).

Coverage mass is concentrated at the high end (mean 81.92%, median 86.14%), consistent with a left tail of failures.

4.1.3 Reward structure and training correlations

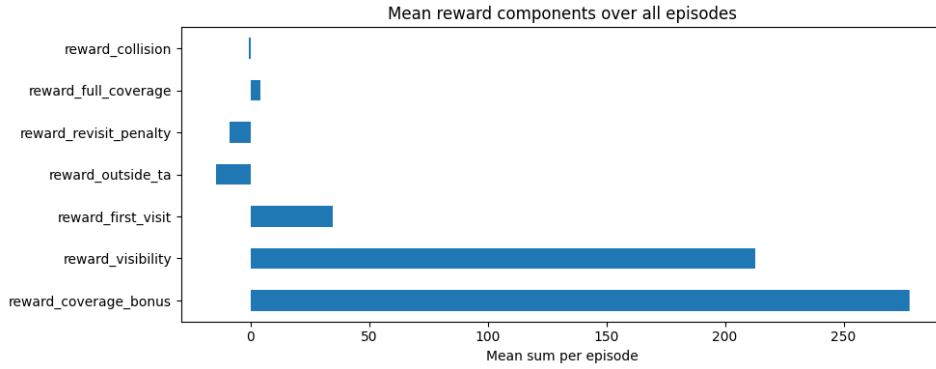


Figure 4.5: Mean reward components over episodes.

The dominant terms are `reward_coverage_bonus` and `reward_visibility`. Penalties (`reward_outside_ta`, `reward_revisit_penalty`) are negative in the mean; `reward_full_coverage` and `reward_collision` are small on average.



Figure 4.6: Coverage versus total reward during training (Pearson $r = 0.947$).

Total reward and final TA coverage are strongly positively associated; the plateau structure at higher return is consistent with discrete coverage-linked bonuses.

4.1.4 Training diagnostics and PPO metrics

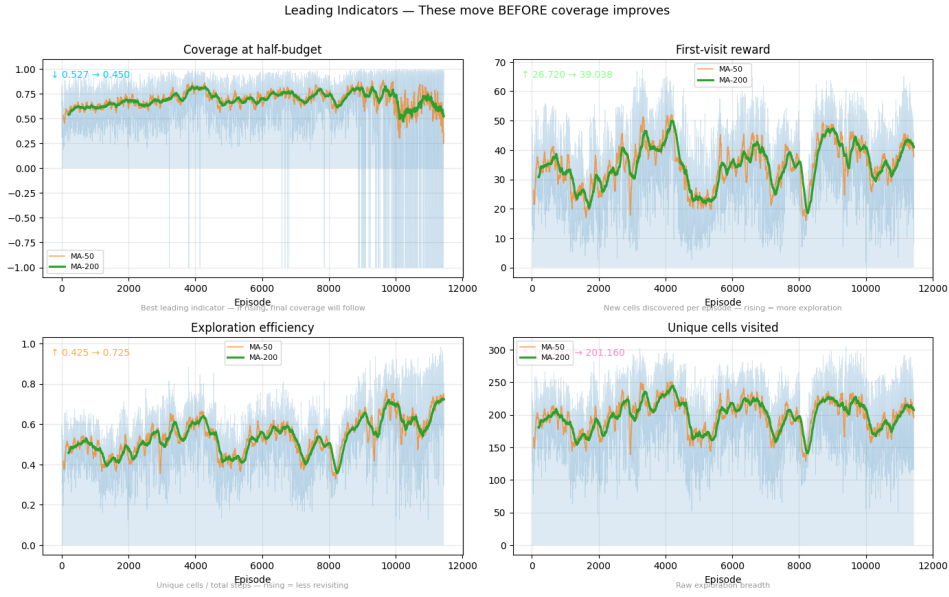


Figure 4.7: Leading indicators versus episode (MA-50 and MA-200).

Figure 4.7 tracks quantities that are informative before final episode coverage is known. *Half-budget coverage* measures how much of the TA has been covered when the average remaining movement budget reaches 50%. In principle, higher half-budget coverage indicates faster early exploration and therefore a larger safety margin for return. The other indicators are more stable: the long-window trend of *first-visit reward* increases, which is consistent with more purposeful movement inside the TA rather than revisits, and *exploration efficiency* improves late in training, meaning the agents visit more unique cells per unit step on average. *Unique cells visited* stabilises around ~ 200 late in training, suggesting the policy has converged to a typical coverage footprint per episode under the fixed budget and constraints.

4 Results



Figure 4.8: Diagnostic indicators (six panels versus episode).

Figure 4.8 provides additional per-episode diagnostics to separate *environment difficulty* from *policy behaviour*.

- **Exploration efficiency:** computed as the number of unique visited cells divided by total executed steps across both agents. This rises overall (with a mid/late dip then recovery), indicating that later policies waste fewer steps on revisits or blocked moves and convert movement into new area coverage more effectively.
- **Steps outside TA:** the count of agent-steps taken outside the target-area footprint. A downward trend in the last third suggests the policy increasingly stays on-task and reduces unproductive travel outside the TA, which also reduces the outside-TA penalty term.
- **Unique cells visited:** a proxy for spatial diversity of motion; oscillations reflect episode-to-episode variability in layout difficulty and in how early forced-return is triggered.
- **NFZ volume fraction:** the fraction of voxels marked as no-fly in the generated layout. Its stability on average indicates that the environment generator is not systematically changing difficulty over time; fluctuations represent random layout draws.

4 Results

- **Spawn-to-TA distance:** the mean distance from spawn (SLZ) to the nearest TA column. Stability here supports the interpretation that improvements in coverage are primarily driven by the learned policy rather than progressively easier starts.
- **Coverage at half budget:** repeats the leading-indicator view.

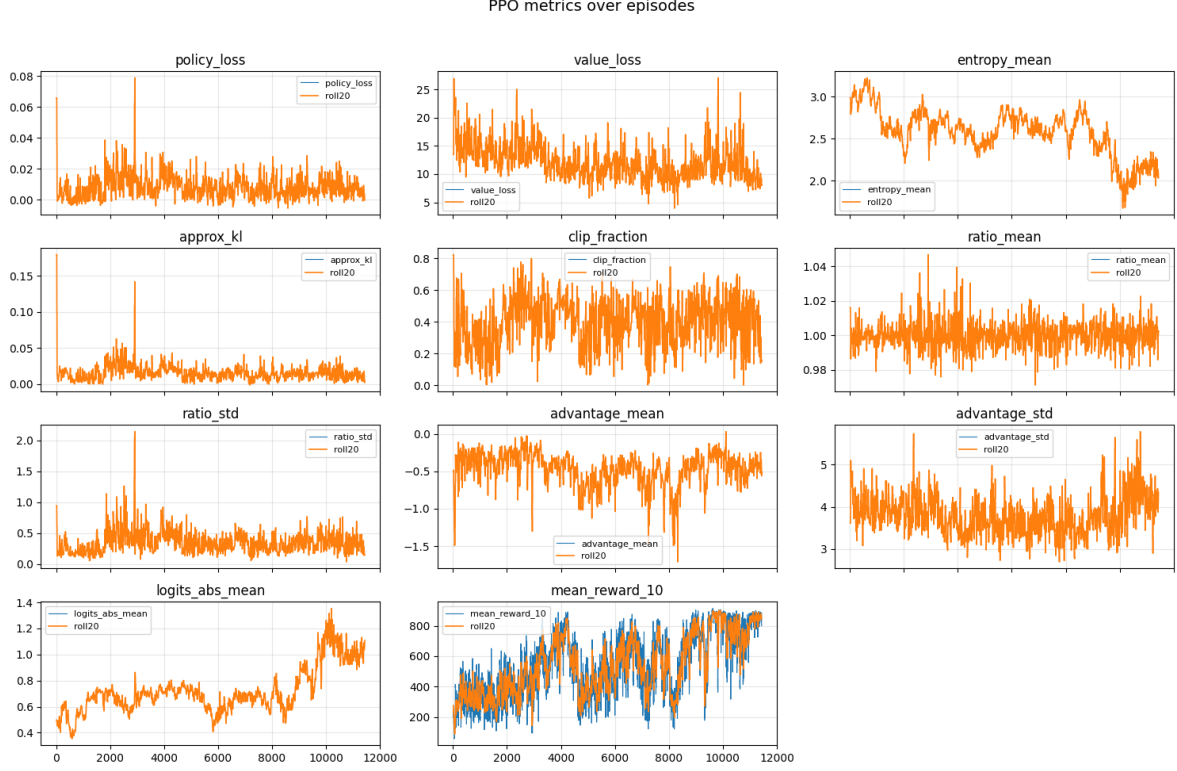


Figure 4.9: PPO and training metrics versus episode.

Figure 4.9 summarises PPO optimisation behaviour alongside a few training-time diagnostics.

- **Short-window mean reward:** computed over a 10-episode window. Its upward trend matches the coverage improvements reported earlier and confirms that the optimiser is making progress under the shaped reward.
- **Entropy:** decreases over training, meaning the policy distribution becomes more peaked (more deterministic). This is expected as PPO converges; it also aligns with the observation that late checkpoints perform well under greedy evaluation.
- **Logits absolute mean:** increases, which indicates the network is producing larger-magnitude action preferences. This typically co-occurs with reduced entropy and a

stronger separation between preferred and non-preferred actions (after legal-action masking).

- **Policy loss:** becomes small after the initial warm-up, suggesting that most updates make incremental changes rather than large policy shifts. This is consistent with PPO’s clipped objective limiting step size.
- **Value loss:** stays mostly within a moderate band, which indicates the critic fits returns reasonably well for most of training. The late spike suggests a transient mismatch between predicted values and realised returns (e.g., due to a distributional change in rollouts or a brief instability), but it does not persist as a sustained divergence.
- **Approximate KL:** remains low most of the time with occasional spikes. Low KL implies the new policy stays close to the behaviour policy used to collect the rollouts; spikes reflect updates where PPO tries to move the policy more aggressively, which is also when clipping becomes more active.
- **Ratio mean and clip fraction:** the probability ratio mean stays near 1, which indicates updates are centred on small changes on average. The clip fraction is volatile because it depends on how many timesteps fall outside the $[1 - \epsilon, 1 + \epsilon]$ trust region; when it rises, a larger portion of the batch is being clipped and the effective step size is being constrained.
- **Advantage mean and std:** the advantage mean is negative but drifts toward 0, and the standard deviation is relatively stable. This suggests that advantage estimates remain well-scaled and do not explode, supporting stable PPO updates during the later part of training.

Overall, the PPO traces are consistent with a stable late-training regime: entropy decays as the policy becomes more decisive, KL remains low most of the time with clipping absorbing occasional aggressive updates, and both policy/value losses stay bounded (aside from brief transients). Together with near-1 ratio mean and steady advantage scale, these signals suggest that the optimisation dynamics remain controlled while performance improves.

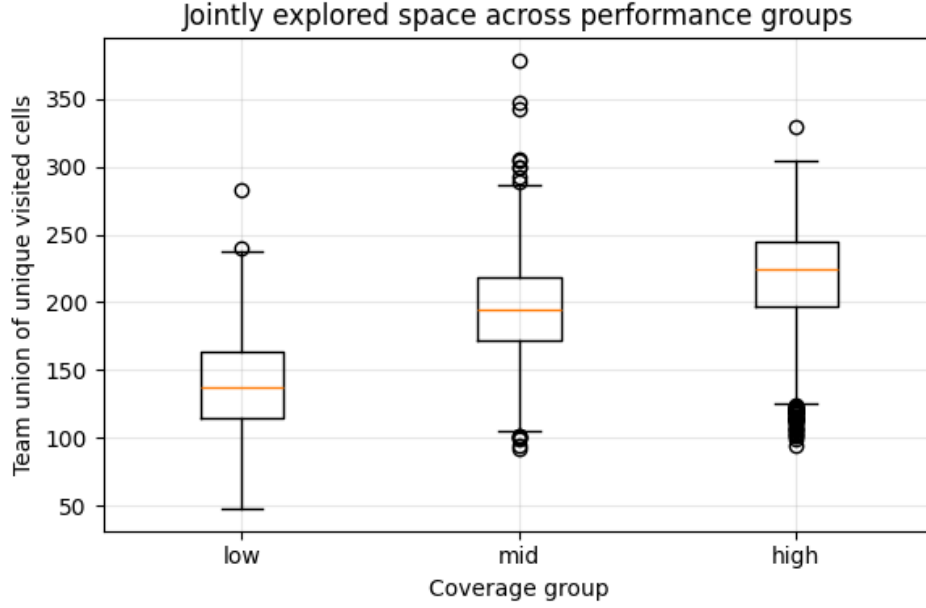


Figure 4.10: Jointly explored space across performance groups. The plotted quantity is the union of unique visited 3D cells across both agents within an episode.

Figure 4.10 shows a clear upward shift in the distribution of jointly explored space from the low-coverage to the high-coverage group. The median and interquartile range increase across the groups, indicating that successful episodes are associated with a larger jointly explored area. This suggests that high final coverage is not only a consequence of local motion efficiency, but also of effective team-level exploration in which the two agents collectively reach more distinct parts of the environment.

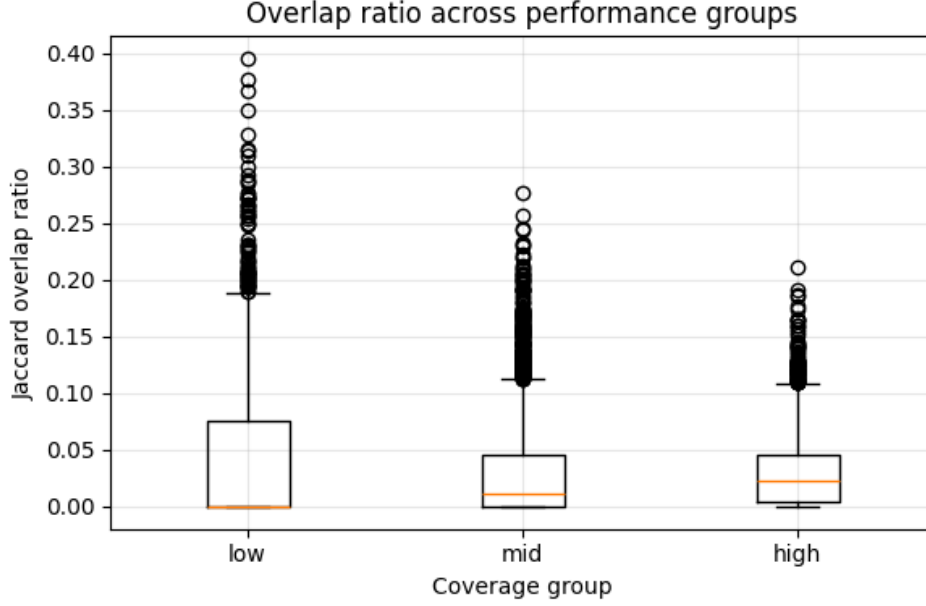


Figure 4.11: Inter-agent overlap ratio across performance groups, measured as the Jaccard overlap between the sets of visited cells of agent 0 and agent 1.

Figure 4.11 complements this view by showing how much the agents revisit the same spatial regions. The overlap ratio remains low overall, which indicates that the learned policy generally avoids heavy redundancy between agents. The high-performance group is concentrated at low overlap values, suggesting that successful episodes are characterised by complementary exploration rather than duplicated motion. Although some overlap is still present, especially in a small number of outlier episodes, the dominant pattern is that strong coverage can be achieved while keeping inter-agent redundancy limited.

Taken together, these two plots provide evidence of emergent coordination. High-performing episodes are associated with a larger jointly explored area and persistently low overlap between the agents, which indicates that the policy tends to divide the exploration burden across agents instead of sending both agents repeatedly into the same regions. In the context of the proposed recurrent Liquid Neural Network policy, this behaviour is consistent with a controller that can adapt its decisions over time based on evolving mission state, rather than relying on fixed hand-designed spatial assignments.

4.2 Checkpoint evaluation (100 fixed seeds; greedy decoding)

The four checkpoint selection rules from Section 3.4.2 are evaluated under identical conditions. Tables use the same short tags as in the methods chapter (`eval_best`, `eval_last_100`, `eval_last_500`, `eval_latest`) to label rows. Each agent is run with either 100 or 150 movement steps per episode; all numbers below are aggregated over 100 evaluation seeds held out from training. A separate comparison of greedy and stochastic action selection uses only the latest saved checkpoint.

4.2.1 Core checkpoint metrics

Table 4.3: Checkpoint comparison (core metrics). Movement budget 100.

Checkpoint	Mean cov.	Std cov.	Min cov.	Mean reward	Std reward	% episodes $\geq$ 90% cov.	Expl. eff.
<code>eval_best</code>	0.836	0.150	0.198	581.93	227.38	42%	0.545
<code>eval_last_100</code>	0.881	0.157	0.242	679.53	243.37	65%	0.664
<code>eval_last_500</code>	0.670	0.211	0.241	349.19	253.14	15%	0.423
<code>eval_latest</code>	0.851	0.129	0.198	596.74	208.57	44%	0.561

Table 4.4: Checkpoint comparison (core metrics). Movement budget 150.

Checkpoint	Mean cov.	Std cov.	Min cov.	Mean reward	Std reward	% episodes $\geq$ 90% cov.	Expl. eff.
<code>eval_best</code>	0.924	0.115	0.198	748.47	184.32	79%	0.511
<code>eval_last_100</code>	0.932	0.118	0.242	763.03	209.80	81%	0.576
<code>eval_last_500</code>	0.727	0.237	0.241	443.15	316.80	36%	0.347
<code>eval_latest</code>	0.922	0.113	0.198	742.96	178.93	78%	0.526

4.2.2 Evaluation thresholds and diagnostic summary

Table 4.5: Movement budget 150: coverage thresholds and hit rates (evaluation, 100 seeds).

Checkpoint	Mean	Std	Min	Max	$\geq 95\%$	$\geq 90\%$	$\geq 85\%$	$\geq 80\%$	$\geq 75\%$	$\geq 70\%$	Collapse
eval_best	0.924	0.115	0.198	0.994	59%	79%	88%	92%	94%	97%	2%
eval_last_100	0.932	0.118	0.242	1.000	70%	81%	88%	90%	94%	96%	2%
eval_last_500	0.727	0.237	0.241	0.991	24%	36%	43%	52%	54%	57%	17%
eval_latest	0.922	0.113	0.198	0.994	51%	78%	87%	93%	96%	97%	2%

Collapse definition. In Tables 4.5 and 4.7, **Collapse** denotes the share of seeds with final coverage $< 45\%$.

Table 4.6: Movement budget 150: task-focus diagnostics (console evaluation, 100 seeds).

Checkpoint	Mean steps outside TA	Mean exploration efficiency
eval_best	84.3	0.511
eval_last_100	162.7	0.576
eval_last_500	240.3	0.347
eval_latest	87.8	0.526

Takeaway (budget 150). The best 100-episode rolling-mean model leads on mean coverage, $\geq 95\%$ and $\geq 90\%$ hit rates, maximum coverage (1.0), and exploration efficiency. The 500-episode rolling-mean model is clearly weaker (17% collapse, many steps outside the TA). Notably, the best rolling-mean model also logs more mean steps outside the TA than the best-episode or latest checkpoints despite achieving the highest coverage.

Table 4.7: Movement budget 100: coverage thresholds and hit rates (console evaluation, 100 seeds).

Checkpoint	Mean	Std	Min	Max	$\geq 95\%$	$\geq 90\%$	$\geq 85\%$	$\geq 80\%$	$\geq 75\%$	$\geq 70\%$	Collapse
eval_best	0.836	0.150	0.198	0.990	19%	42%	61%	74%	82%	85%	4%
eval_last_100	0.881	0.157	0.242	0.997	48%	65%	75%	81%	85%	88%	3%
eval_last_500	0.670	0.211	0.241	0.991	9%	15%	26%	34%	39%	48%	18%
eval_latest	0.851	0.129	0.198	0.991	18%	44%	60%	76%	86%	90%	3%

Table 4.8: Movement budget 100: task-focus diagnostics (evaluation, 100 seeds).

Checkpoint	Mean steps outside TA	Mean exploration efficiency
eval_best	58.3	0.545
eval_last_100	58.9	0.664
eval_last_500	132.4	0.423
eval_latest	59.3	0.561

Takeaway (budget 100). The ranking matches budget 150: the 100-episode rolling-mean checkpoint again leads on the high-coverage thresholds. The 500-episode rolling-mean checkpoint shows 18% collapse and roughly 132 steps outside the TA on average, compared with about 58–59 for the other three. Episodes with very high final coverage are rarer than at budget 150 (for the rolling-mean leader, 48% above 95% here versus 70% in Table 4.5).

4.2.3 Greedy versus stochastic action selection (latest checkpoint)

Movement budget: 150 steps per agent

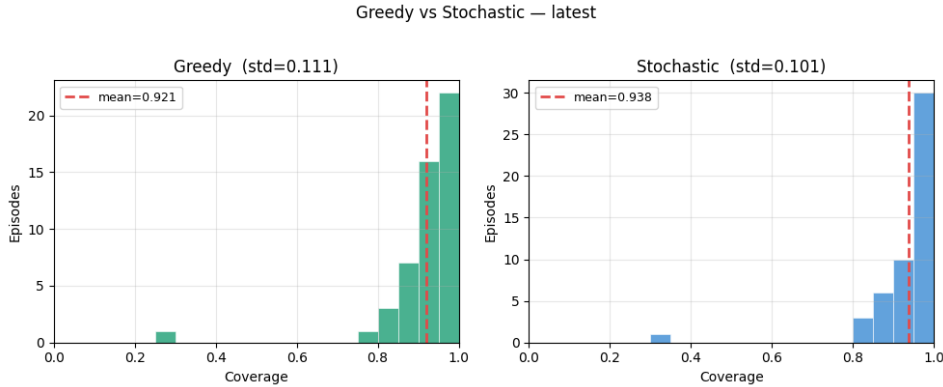


Figure 4.12: Greedy versus stochastic coverage histograms (latest checkpoint, budget 150; 50 seeds).

Mass is near 1.0 with one low outlier per mode (~ 0.25 – 0.35). Stochastic has higher mean (0.938 vs 0.921) and lower standard deviation (0.101 vs 0.111). The greedy mean approximately matches the latest-checkpoint aggregate in Table 4.5 up to rounding.

4 Results

Table 4.9: Greedy versus stochastic (latest checkpoint, budget 150; 50 episodes).

Mode	Episodes	Mean	Std	Min	Max	$\geq 95\%$	$\geq 90\%$	$\geq 85\%$	$\geq 80\%$	$\geq 75\%$	$\geq 70\%$	Collapse
Greedy	50	0.921	0.111	0.255	0.997	44%	76%	90%	96%	98%	98%	2%
Stochastic	50	0.938	0.101	0.343	1.000	60%	80%	92%	98%	98%	98%	2%

Table 4.10: Task-focus diagnostics for greedy versus stochastic (latest checkpoint, budget 150; same 50-episode batch).

Mode	Mean steps outside TA	Mean exploration efficiency
Greedy	86.9	0.494
Stochastic	72.4	0.692

Movement budget: 100 steps per agent

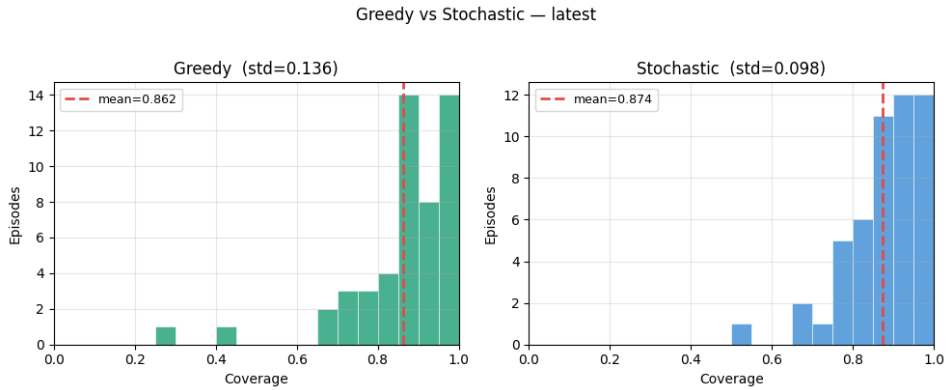


Figure 4.13: Greedy versus stochastic coverage histograms (latest checkpoint, budget 100; 50 seeds).

Bulk coverage is in the 0.8–1.0 range; greedy shows deeper low tails. Annotated means match the table below and align approximately with the 100-seed latest-checkpoint aggregate in Table 4.7.

Table 4.11: Greedy versus stochastic summary (latest checkpoint, budget 100; 100 seeds, histogram annotations).

Mode	Mean coverage	Std
Greedy	0.862	0.136
Stochastic	0.874	0.098

Table 4.12: Greedy versus stochastic (latest checkpoint, budget 100; 50 episodes).

Mode	Episodes	Mean	Std	Min	Max	$\geq 95\%$	$\geq 90\%$	$\geq 85\%$	$\geq 80\%$	$\geq 75\%$	$\geq 70\%$	Collapse
Greedy	50	0.862	0.136	0.255	0.991	28%	44%	72%	80%	86%	92%	4%
Stochastic	50	0.874	0.098	0.513	0.993	24%	48%	70%	82%	92%	94%	0%

Table 4.13: Task-focus diagnostics for greedy versus stochastic (latest checkpoint, budget 100; same 50-episode batch).

Mode	Mean steps outside TA	Mean exploration efficiency
Greedy	59.3	0.542
Stochastic	46.2	0.729

4.3 Overall summary

- **Training:** Rolling and staged coverage improve; late windows are near 97% mean coverage. The histogram shows heavy high-coverage mass with a left tail. The reward signal is dominated by the coverage milestone bonus and visibility-based reward terms; coverage and return correlate (Pearson $r \approx 0.95$) with plateau structure. Leading indicators and diagnostics show rising efficiency and stable environment and spawn statistics. PPO curves show learning with typical variability.
- **Checkpoints:** The checkpoint chosen by the best 100-episode rolling mean attains the strongest mean coverage, return, and exploration efficiency at both budgets; the 500-episode rolling-mean save is weakest, with higher collapse rates and more steps outside the TA.

- **Greedy versus stochastic** (latest checkpoint): In the reported runs, stochastic evaluation improves mean coverage and reduces variance at both budgets; the 50-episode tables show higher exploration efficiency and fewer steps outside the TA under stochastic sampling.

5 Discussion

5.1 Overview

This chapter interprets the results in Chapter 4 in relation to the four research questions. It connects the findings to the literature reviewed in Chapter 2 and clarifies what the evidence supports. Chapter 4 reported numbers; here I explain what they mean for the proposed LTC-PPO system.

5.2 RQ1: Can LTC-PPO learn effective 3D multi-agent CPP?

Yes. Under procedural terrain, NFZ, movement budgets, and two agents, the trained policy reached high target-area coverage in training and transferred that behaviour to fixed evaluation seeds that were not seen during training. The strongest quantitative support is the checkpoint evaluation: the model retained under the best 100-episode rolling-mean rule achieved mean TA coverage of about 0.93 at a 150-step budget and 0.88 at a 100-step budget on 100 fixed seeds (Tables 4.4 and 4.3), with a large share of episodes above 90% coverage (Tables 4.5 and 4.7). Training curves show the 100-episode rolling mean coverage moving from the mid-0.5 range toward ~ 1.0 , with trailing windows averaging $\sim 97\%$ mean coverage (Figures 4.1 and 4.2 and Table 4.1). Exploration efficiency increased over the run and was highest for that rolling-mean checkpoint relative to the other saved models, which matches the idea that the policy learned to obtain more new TA coverage per joint step as competence improved. Half-budget coverage and the other diagnostics (Figures 4.7 and 4.8) are noisier, but they are consistent with mid-episode progress becoming stronger before final coverage as training advances along the episode index.

The strong correlation between final TA coverage and total return in training (Figure 4.6) is what I would expect if the reward is doing its job: most of the return is tied to coverage milestones and visibility-based discovery, with penalties for wasteful revisits and time outside the TA. The step-like structure at high return reflects discrete milestone bonuses; it does not weaken the link between good coverage and high return.

Three points link the results to why LTC was a reasonable choice for this problem.

Trajectory memory. Coverage path planning is not a purely reactive task. The agent must remember where it has already been, avoid useless revisits, and spread work across the episode under a hard budget. The recurrent core is there to carry information across steps; the rise in exploration efficiency from weaker checkpoints to `eval_last_100` is consistent with the policy learning to use temporal context so that movement translates into durable progress on the map rather than short bursts of reward.

Multi-timescale observations. The observation mixes fast-changing quantities (position, local occupancy, legal actions) with quantities that change slowly (accumulated coverage, remaining budget, return pressure). LTC’s input-adaptive dynamics are motivated in the literature for settings where different parts of the state evolve on different time scales. My results are compatible with that motivation: they do not isolate LTC from a fixed-rate RNN, but they show that the combined architecture can learn a strong policy in practice.

Generalisation to new seeds. Evaluation used 100 fixed seeds per checkpoint and per budget, drawn separately from training. High mean coverage on that suite is therefore not explained by memorising training maps. This aligns directionally with robustness arguments for recurrent policies in the literature (for example, work that motivates LTC-style models).

Overall, these results support the feasibility of the proposed LTC–PPO system for this 3D multi-agent CPP setup: the best rolling-mean checkpoint delivers high target-area coverage and strong exploration efficiency on held-out fixed seeds. This chapter focuses on the behaviour of the full system under the chosen reward and training configuration, rather than isolating which architectural component is primarily responsible.

5.3 RQ2: Does attention-based LTC coordination produce safe cooperative behaviour?

Yes, in the sense captured by my metrics: no collision penalties in evaluation, and balanced forced-return coverage between agents.

Collisions. The reward includes an explicit collision penalty for same-cell conflicts and swap attempts (Chapter 3). I did not add a separate motion planner for avoidance; safety has to emerge from the policy and the reward. In the fixed-seed evaluation logs used for the four checkpoints (`eval_best`, `eval_last_100`, `eval_last_500`, `eval_latest`), **reward_collision is zero on every episode** in both the 100- and 150-step budget folders (800 episodes in total across checkpoints and budgets). That is a clean result for the evaluation protocol I used. Training logs are different: some training episodes still incur collision penalties, which reflects stochastic action sampling and occasional conflicting proposals between agents before the policy fully sharpens avoidance behaviour.

Balance between agents. For cooperative CPP, one failure mode is one agent doing almost all useful coverage while the other idles. The environment logs forced-return coverage per agent. On the 100-seed run for `eval_last_100` at budget 100, mean forced-return coverage is about 0.871 for agent 0 and 0.867 for agent 1, with mean absolute difference about 0.005. That is a small gap and suggests a roughly balanced division of work in that checkpoint, not one dominant and one passive agent.

Interpretation. Two explanations fit the data, and I cannot separate them with the present experiment. First, reward design: the collision term, outside-TA penalty, shared coverage objective, and visibility shaping may already push the policy toward non-overlapping, on-task behaviour. Second, architecture: attention lets each agent’s decision depend on the other agent’s representation, and the LTC state carries temporal context, so coordination may be partly learned through representation rather than only through scalar rewards. The most defensible interpretation of the evidence is: the results are consistent with emergent coordination under the attention-based recurrent policy, but they do not disentangle reward effects from architectural effects.

5.4 RQ3: Which checkpoint criterion is most reliable?

In this experimental setting, the 100-episode rolling mean (`eval_last_100`) was the most reliable selector among the four I tested. It achieved the highest mean coverage, highest mean return, and highest mean exploration efficiency at both movement budgets (Tables 4.4 and 4.3), and the strongest threshold hit rates in the summaries (Tables 4.5 and 4.7).

That outcome differs from the *a priori* expectation that a longer window always dominates a shorter one. The 500-episode rolling-mean save (`eval_last_500`) was the weakest on evaluation: lower mean coverage, much higher collapse rate (for example 17–18% of seeds below 45% coverage versus 2–4% for the others), and the worst exploration efficiency. Thus a longer rolling average did not yield a better checkpoint in this run.

The single best-episode save (`eval_best`) is still useful as a sanity check: it underperformed `eval_last_100` on mean coverage and reward at both budgets. That supports the general methodological point that one lucky training episode is a risky selection rule in a stochastic environment.

The latest save (`eval_latest`) was close to `eval_best` on means but below `eval_last_100`. Thus simply training for longer and using the final weights was not optimal in this run.

Practical takeaway. For stochastic CPP with similar logging, I would recommend preferring a rolling-mean criterion over a single-episode peak, and validating the window length on held-out seeds rather than assuming 500 episodes beats 100. In my results, 100 episodes was the better window.

5.5 RQ4: Is PPO training stable with LTC policies?

Yes, based on the training metrics reported in Chapter 4. The PPO diagnostic panel (Figure 4.9) shows the short-window mean reward trending upward, entropy remaining positive (drifting down but not collapsing to zero), policy loss small after the initial segment, value loss mostly in a moderate band with a late excursion, approximate KL mostly in a modest range with occasional spikes, and ratio statistics near unity.

What that suggests in plain terms:

- **Entropy** did not collapse to a near-deterministic policy too early; there was still stochasticity in the policy.
- **KL** staying mostly bounded is consistent with updates staying inside a trust-region style regime, aside from short spikes.
- **No skipped updates** points to numerical and optimisation stability for the chosen settings.
- **Advantage** statistics (mean negative but drifting toward zero over the second half of training, with fairly stable standard deviation) are in line with a critic that tracks a shifting return scale rather than a diverging value runaway, interpreted cautiously without a formal stability proof.

The LTC core is trained with truncated BPTT over long rollouts, so a reasonable concern is whether PPO’s policy gradient steps interact badly with backprop through the recurrent dynamics. The observed learning curve and PPO scalars are consistent with compatibility between PPO and this LTC implementation under the fixed hyperparameters in Chapter 3.

5.5.1 Role and importance of liquid neural networks in multi-agent coverage

An important question raised by the results is why a Liquid Neural Network (LNN) is a suitable policy class for the considered multi-agent coverage path planning problem. The relevance of the LNN does not lie only in its ability to approximate a nonlinear policy, but more importantly in its ability to represent temporally evolving decision processes. In the present task, the usefulness of an action depends not only on the current observation, but also on the prior exploration history, the remaining movement budget, the current mission phase, and the evolving behaviour of the other agent. This makes the problem inherently sequential and history-dependent, rather than purely reactive.

The empirical results are consistent with this interpretation. Training converged toward a strong late-episode operating region with high rolling mean coverage, bounded optimisation signals, and strong checkpoint performance under greedy evaluation. In addition, the trajectory-based coordination analysis showed that high-performing episodes are

associated with a larger jointly explored area, low inter-agent overlap, and more balanced contribution between agents. These behavioural patterns suggest that the learned policy does not merely react locally to immediate observations, but instead produces structured exploration over the course of an episode. Such behaviour is well aligned with the intended strengths of recurrent liquid dynamics, which maintain and update an internal state while the mission unfolds.

This temporal aspect is particularly important in the present environment. The agents must not only search for uncovered target-area cells, but must also adapt continuously to terrain structure, no-fly zones, and the return-to-base constraint imposed by limited movement budget. The same local observation can therefore require different actions depending on context. For example, a frontier cell that is worth exploring early in the episode may no longer be worth pursuing when the return phase approaches. Similarly, an area that appears attractive from a purely local perspective may be less useful if it has likely already been explored by the other agent. A recurrent architecture with adaptive hidden dynamics is therefore more appropriate than a purely feed-forward mapping, because it can condition the current action on an evolving internal representation of mission progress.

The coordination results support this view. The agents generally maintain low overlap in visited cells while jointly exploring a larger area in successful episodes, which indicates complementary exploration rather than repeated duplication of the same motion. Importantly, this behaviour emerged without manually defined spatial partitions or explicit hand-crafted roles such as assigning one agent to the left half of the map and the other to the right half. From a practical perspective, this is one of the most relevant advantages of the learned approach: coordination arises through interaction and optimisation, rather than through rigid task decomposition. In this sense, the LNN-based controller appears capable of supporting flexible and adaptive cooperation in a changing 3D environment.

Another point of importance is robustness under varying layouts. Since terrain, target-area shape, and no-fly-zone placement vary across episodes, the underlying control problem is non-stationary. A policy that performs well in this setting must adapt online rather than rely on memorised spatial templates. The observed performance across training and evaluation suggests that the proposed controller can handle this variability to a substantial degree. This is again consistent with the motivation for using liquid neural dynamics, which are intended to remain responsive to changing inputs over time.

The present results show that the proposed LNN-based policy is capable of learning coordinated and efficient behaviour.

Overall, the results suggest that the importance of the Liquid Neural Network in this work lies in its compatibility with the demands of the problem: temporally extended decision-making, adaptive control under changing constraints, and the emergence of coordination without hard-coded role assignment. The behavioural evidence from coverage performance, trajectory overlap, and jointly explored space is consistent with the view that liquid recurrent dynamics provide a strong architectural match for multi-agent 3D coverage path planning.

5.6 Future work

The experiments in this thesis were designed to establish feasibility and a reproducible baseline. Several directions would strengthen claims about *why* LTC is preferable to alternatives, how far the approach scales, and how conclusions generalise beyond the present simulator.

Comparison with other temporal models. The LTC recurrent core was chosen for its input-adaptive dynamics, but the present study does not report head-to-head results against other sequence models under identical encoders, rewards, and compute budgets. Future work should include a systematic comparison with widely used temporal backbones, for example vanilla RNNs, LSTM, and GRU [HS97; Chu+14] and, if compute permits, alternatives such as temporal convolutional stacks or Transformer-style sequence layers over the per-step feature sequence [Vas+17]. Each candidate should share the same Conv3D and scalar branches, action masking, PPO schedule, and evaluation seeds so that differences in coverage, stability, and sample efficiency can be attributed to the temporal module rather than training noise. Reporting wall-clock inference time and memory per step would also clarify whether LTC’s behaviour comes at a cost in onboard deployability relative to lighter recurrent cells.

Other architectural ablations. Beyond temporal models, the results do not isolate attention-based communication or the multi-branch encoder. A memoryless feed-forward policy (or a shallow history via stacked frames) would bound how much recurrent depth

is necessary for this task. Turning off or replacing the inter-agent attention block would test how much coordination is explained by the communication module versus shared rewards alone.

Hardware deployment and implementation. Translating the trained policy from simulation to physical UAVs requires an implementation path that this thesis does not exercise. Practical next steps include exporting the network for onboard inference (for example TorchScript, ONNX, or vendor runtimes), measuring end-to-end latency and power draw on flight-grade computers, and integrating the policy with an autopilot or low-level controller via middleware such as ROS 2 or MAVLink so that discrete grid actions map to feasible motion primitives or waypoint updates. Multi-agent deployment additionally needs time-synchronised telemetry and conflict-free communication within radio and regulatory limits. Hardware-in-the-loop benches and confined flight tests would validate timing and safety before open-area trials; a parallel software stack should retain deterministic fallbacks (for example geofencing, manual override, and watchdogs on budget and NFZ violations) because learned policies do not provide formal guarantees on their own.

Scaling agents, maps, and dynamics. The environment uses two agents and a fixed grid extent. Extending the formulation to larger teams, larger volumes, or finer resolution would test sample efficiency and coordination complexity. Continuous-time or dynamics-level UAV models (kinematic or low-level controllers), wind, or sensing noise would move the setting closer to deployment while preserving the same high-level coverage objective.

Real-world transfer and sensing. The visibility model and voxel grid are simplifying assumptions. Closing the gap to hardware also requires replacing the idealised grid with streams from lidar, cameras, or downward-facing sensors, injecting latency and dropout between sensing and actuation, and quantifying performance loss relative to simulation. Field trials (where airspace rules and safety cases allow) would complement the deployment considerations above.

5.7 Closing remarks

Taken together, the results support the central claim of this thesis: an LTC-based PPO policy can learn effective, stable, and coordinated behaviour for constrained 3D multi-agent coverage path planning. Across training and fixed-seed evaluation, the proposed system achieved high target-area coverage, maintained safe behaviour under the chosen collision metric, and produced trajectory patterns consistent with complementary multi-agent exploration rather than redundant motion. These findings are especially meaningful because they were obtained in a procedurally varying environment with terrain, no-fly constraints, movement budgets, and return budget pressure, rather than in a trivial static setting.

Beyond demonstrating strong performance, this thesis also provides a useful interpretation of why the approach is suitable for the task. The results are consistent with the view that recurrent LTC dynamics are a good architectural match for multi-agent coverage problems in which decisions depend on mission history, remaining budget, and the evolving behaviour of other agents. In that sense, the contribution is not only empirical but also conceptual: the thesis shows how Liquid Neural Network-based policies can be framed as temporally adaptive controllers for cooperative 3D coverage.

A further contribution of the work is methodological. The checkpoint analysis showed that the 100-episode rolling mean was a more reliable selection rule than either a single best episode, the latest checkpoint, or a much longer 500-episode rolling mean in this setting. This is a practically relevant result for future reinforcement-learning studies on stochastic coverage tasks, where apparent training peaks do not necessarily translate into the most robust evaluation performance.

Overall, this work moves beyond a proof-of-concept. It establishes that an LTC-based controller within the Liquid Neural Network family can achieve high-performance cooperative coverage under realistic operational constraints. Section 5.6 outlines concrete directions for comparative evaluation, scaling, and stronger statistical validation.

6 Conclusion

This thesis investigated whether a Liquid Neural Network approach, instantiated as an LTC-based recurrent policy trained with PPO, can solve constrained 3D multi-agent coverage path planning under terrain, no-fly zones, and hard movement-budget safety constraints. The central finding is that this integrated LTC-PPO pipeline is feasible and effective in the studied setting.

Across 11,436 training episodes, coverage and reward improved consistently, with a strong positive association between return and final target-area coverage. Late-run training windows were near 97% mean coverage, while held-out fixed-seed evaluation confirmed robust performance. Among the four checkpoint criteria, the model saved at the best 100-episode rolling mean of return was the most reliable in this study, achieving the strongest overall results at both tested budgets (approximately 0.93 mean coverage at budget 150 and 0.88 at budget 100), together with favourable threshold hit rates and exploration efficiency.

The results also support the behavioural claims made in the discussion chapter. The policy exhibited safe interaction under the chosen metrics (no collision penalties across the reported fixed-seed checkpoint evaluations) and showed coordination patterns consistent with complementary exploration, including low overlap and larger jointly explored space in stronger episodes. PPO diagnostics remained in a regime consistent with stable optimisation for the reported configuration.

Overall, this thesis establishes a concrete baseline for LiquidNN/LTC-based CPP and provides a reproducible foundation for deeper benchmarking. Detailed directions for follow-on work including comparisons with other temporal models, hardware deployment, scaling, statistical reporting, and transfer are summarised in Section 5.6.

A Software implementation and algorithms

A.1 Software stack

The experiments were implemented in Python using PyTorch for the policy and optimisation loop. The Liquid Time-Constant (LTC) recurrent core uses the `ncps` library (Neural Circuit Policies / AutoNCP wiring). The training environment follows a Gymnasium-style interface: procedural resets, vectorised observations, discrete action spaces with masking, and step-wise logging suitable for PPO rollouts.

A.2 Modular structure

At the level of this thesis, the implementation separates (i) **environment dynamics** (grid, terrain, NFZ, target area, visibility, budgets, forced return), (ii) **policy forward pass** (multi-branch features, LTC state, attention-based inter-agent communication, masked logits and value), and (iii) **PPO training** (trajectory collection, advantage estimation, clipped updates, truncated backpropagation through time). Checkpoint files store model parameters together with a configuration record so that evaluation can rebuild the environment and policy with matched hyperparameters.

A.3 Target-area generation

After sampling terrain heights $h(x, y)$ and constructing the NFZ volume for an episode, the target area (TA) is derived as the *surface layer* above the terrain (Chapter 3).

Algorithm 1 states the per-column rule and the 2D footprint used for inside-TA tests. If the NFZ removes an excessive fraction of TA voxels, the implementation applies a repair step that unblocks selected NFZ cells inside the TA so the task does not collapse to a trivial instance.

Algorithm 1 Target-area (TA) voxel layer from terrain (per episode).

- 1: **Inputs:** grid dimensions (H, W, D) ; integer terrain height $h(x, y)$ for each column; NFZ mask (optional repair against excessive TA removal)
 - 2: **Output:** binary TA mask on voxels; 2D TA footprint $\mathcal{F} \subseteq \{0, \dots, H - 1\} \times \{0, \dots, W - 1\}$
 - 3: **for** each horizontal coordinate (x, y) with valid terrain **do**
 - 4: $z_{\text{ta}} \leftarrow \min(D - 1, h(x, y) + 1)$ $\triangleright$ single TA voxel per column, one cell above terrain
 - 5: mark voxel (x, y, z_{ta}) as belonging to the TA
 - 6: add (x, y) to footprint $\mathcal{F}$
 - 7: **end for**
 - 8: Adjust NFZ–TA overlap if coverage would otherwise be degenerate (see main text)
-

A.4 Shortest-path distances and forced return (BFS)

Return distance and the forced-return policy use **breadth-first search (BFS)** on the same 6-connected grid as agent motion: edges connect unit steps along $\pm x, \pm y, \pm z$ between *flyable* voxels (strictly above terrain, outside NFZ, within bounds). For the distance feature to the nearest safe landing zone (SLZ), other agents that have not landed are treated as blocking obstacles, matching the methods description. Algorithm 2 computes the minimum number of steps from an agent position to any SLZ cell. The same graph search, storing predecessor pointers, yields a shortest path; in forced-return mode the environment follows the first hop of that path when the remaining movement budget falls below the required return budget plus buffer.

Algorithm 2 Shortest path length to the nearest SLZ (6-connected BFS).

```

1: Inputs: start voxel  $v_0$ ; set of SLZ cells  $\mathcal{S}_{\text{SLZ}}$ ; flyability predicate (bounds, terrain,
   NFZ, dynamic obstacles)
2: Output: step count  $d$  to the nearest SLZ, or  $\infty$  if unreachable
3: Initialise empty queue  $Q$ ; enqueue  $v_0$  with distance 0; mark  $v_0$  visited
4: while  $Q$  is not empty do
5:   dequeue front cell  $u$  with label  $\text{dist}(u)$ 
6:   if  $u \in \mathcal{S}_{\text{SLZ}}$  then
7:     return  $\text{dist}(u)$ 
8:   end if
9:   for each 6-connected neighbour  $w$  of  $u$  do
10:    if  $w$  is flyable and not visited then
11:      mark  $w$  visited; enqueue  $w$  with distance  $\text{dist}(u) + 1$ 
12:    end if
13:  end for
14: end while
15: return  $\infty$   $\triangleright$  no SLZ reachable under current obstacles

```

Comparing the remaining budget to the BFS distance (plus a fixed buffer) decides whether to switch an agent to forced-return mode; executing the precomputed shortest path is then equivalent to repeatedly stepping along BFS predecessors until an SLZ is reached.

A.5 High-level training procedure

Algorithm 3 summarises the on-policy loop used in the reported runs: collect one or more episodes per update, compute advantages (here: generalised advantage estimation), and apply several epochs of clipped PPO updates. The hidden state of the LTC is carried across environment steps within an episode and reset at episode boundaries; gradients through the recurrent core use TBPTT with a fixed chunk size during optimisation (see Chapter 3).

Algorithm 3 High-level PPO training step with a recurrent policy (conceptual).

- 1: **Inputs:** policy π_θ , value head V_θ , optimiser, environment $\mathcal{E}$, hyperparameters $(\gamma, \lambda_{\text{GAE}}, \epsilon, K, \dots)$
 - 2: **Output:** updated parameters θ
 - 3: Initialise or load θ ; set LTC hidden state $h \leftarrow 0$ at each episode start
 - 4: Collect rollout(s) from $\mathcal{E}$ using π_θ (with action masking); store $(s_t, a_t, r_t, \text{mask}_t, \dots)$ and terminal flags
 - 5: Compute advantages A_t and returns using GAE and V_θ (bootstrap at truncation if needed)
 - 6: **for** epoch = 1 to K **do**
 - 7: Evaluate π_θ , V_θ on stored sequences; form clipped surrogate loss, value loss, entropy bonus
 - 8: Backpropagate total loss through TBPTT chunks; clip gradients; optimiser step
 - 9: **end for**
 - 10: Save checkpoint under selection criteria (best episode, rolling means, latest)
-

This pseudocode is descriptive: exact tensor shapes, masking of invalid actions, and device placement follow the implementation described in the methods chapter and the accompanying source code.

B Mathematical notation and supplementary formulas

B.1 Notation

Table B.1 collects symbols that recur in the background and methods chapters. Bold vectors denote quantities that vary with time or agent index when stated in the main text.

Table B.1: Frequently used notation.

Symbol	Meaning
$\mathcal{S}, \mathcal{A}$	State and action spaces (MDP formalism).
$\pi_{\theta}(a \mid s)$	Stochastic policy with parameters θ .
$V_{\theta}(s), Q^{\pi}(s, a)$	State value and action-value functions under policy π .
γ	Discount factor.
λ_{GAE}	GAE bias–variance parameter.
A_t	Advantage estimate at time t .
$r_t(\theta)$	PPO probability ratio $\pi_{\theta}(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$.
ϵ	PPO clip range.
h	Local crop half-width around an agent (spatial features).
$m, u, s_{\text{base}}(u), \Delta s$	Master seed, PPO update index, layout base seed, and stride (training protocol).
TA, NFZ, SLZ	Target area, no-fly zone, safe landing zone.

B.2 Generalised advantage estimation

Let $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ denote the one-step TD residual with a learned value function V . The GAE advantage can be written as

$$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l},$$

which reduces variance compared with a pure Monte Carlo return while controlling bias through λ (see Chapter 2).

B.3 PPO clipped objective (scalar form)

For collected trajectories, PPO maximises an expected clipped objective. With advantages A_t and ratio $r_t(\theta)$, a standard form is

$$L^{\text{clip}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)],$$

combined with a value-function loss and entropy term in the total loss (coefficients as in Chapter 3).

C Data artefacts, logs, and evaluation protocol

C.1 Training logs

Each training episode produces a scalar summary row (typically appended to a CSV file) containing at least: episode index, total return, final target-area coverage, episode length, rolling means over recent episodes, optional reward-term aggregates, timing splits (collection vs. optimisation), and PPO diagnostics from the last optimisation epoch (policy loss, value loss, entropy, approximate KL, clip fraction, ratio statistics). These rows are the basis for the learning curves, histograms, and correlation analyses in Chapter 4. The episode count and scope of the reported run are stated in that chapter; the logging schema is fixed across the full training log so that every row is comparable along the episode index.

C.2 Checkpoints

Saved checkpoints bundle (i) policy (and, if applicable, optimiser) state tensors, and (ii) a serialised configuration: grid dimensions, number of agents, LTC sizes, PPO hyperparameters, TBPTT chunk size, and other settings needed to reconstruct the environment and network for evaluation. Checkpoint **selection** (best episode, rolling means, latest) is applied during training; the corresponding files are then evaluated on held-out seeds as described in the methods chapter.

C.3 Evaluation suites

Held-out evaluation uses a fixed set of random seeds (distinct from the training seed schedule) and reports aggregates over those episodes for each saved model and each movement-budget setting. Greedy and stochastic action modes, where reported, use the same checkpoint and seed suite so that differences reflect sampling mode rather than model identity.

C.4 Figures and provenance

Figures in the results chapter were generated from the logged training run and from the evaluation exports associated with the saved checkpoints. Exact plotting scripts reside with the project code; the thesis text specifies the metrics and windowing (e.g. rolling means, stage splits) used in each figure caption.

Declaration on the Use of Artificial Intelligence Assistance

In accordance with the examination regulations of the University of Passau, I hereby declare the following regarding the use of artificial intelligence (AI) assistance in the preparation of this thesis.

AI-based writing assistance were used exclusively for the following purposes: proofreading and grammatical correction of drafted text, improving the clarity and academic style of written passages, identifying formatting and cross-reference inconsistencies in the Latex source, and suggesting alternative phrasings for improved readability.

All research questions, experimental designs, implementation decisions, empirical results, and intellectual conclusions presented in this thesis are entirely my own. The AI assistance did not contribute to the conception of the research, the design or execution of experiments, the generation or analysis of results, or any substantive academic content.

All AI-assisted revisions were critically reviewed, evaluated, and either accepted or rejected by me. I take full responsibility for the final content of this thesis.

Bibliography

- [Ali+25] Mirza Aqib Ali et al. “Comparative Evaluation of Reinforcement Learning Algorithms for Multi-Agent Unmanned Aerial Vehicle Path Planning in 2D and 3D Environments”. In: *Drones* 9.6 (2025), p. 438. DOI: 10.3390/drones9060438 (cit. on p. 13).
- [Aru+17] Kai Arulkumaran et al. “A Brief Survey of Deep Reinforcement Learning”. In: *IEEE Signal Processing Magazine* 34.6 (2017), pp. 26–38 (cit. on p. 8).
- [Bay+21] Harald Bayerlein et al. “Multi-UAV Path Planning for Wireless Data Harvesting With Deep Reinforcement Learning”. In: *IEEE Open Journal of the Communications Society* 2 (2021), pp. 1171–1187. DOI: 10.1109/OJCOMS.2021.3081996 (cit. on p. 13).
- [BD22] Julian Bialas and Mario Döller. “Coverage Path Planning for Unmanned Aerial Vehicles in Complex 3D Environments with Deep Reinforcement Learning”. In: *2022 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2022, pp. 1080–1085. DOI: 10.1109/ROBIO55434.2022.10011936 (cit. on pp. 13, 18, 22).
- [CBF19] Tauã M. Cabreira, Lisane B. Brisolara, and Paulo R. Ferreira Jr. “Survey on Coverage Path Planning with Unmanned Aerial Vehicles”. In: *Drones* 3.1 (2019), p. 4. DOI: 10.3390/drones3010004 (cit. on pp. 1, 18, 24).
- [Can+21] Lorenzo Canese et al. “Multi-Agent Reinforcement Learning: A Review of Challenges and Applications”. In: *Applied Sciences* 11.11 (2021), p. 4948. DOI: 10.3390/app11114948 (cit. on p. 9).
- [Che+18] Ricky T. Q. Chen et al. “Neural Ordinary Differential Equations”. In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018 (cit. on p. 14).

- [Cho01] Howie Choset. “Coverage for Robotics—A Survey of Recent Results”. In: *Annals of Mathematics and Artificial Intelligence* 31 (2001), pp. 113–126 (cit. on pp. 1, 6).
- [Chu+14] Junyoung Chung et al. “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling”. In: *arXiv preprint arXiv:1412.3555* (2014) (cit. on pp. 15, 65).
- [Foe+18] Jakob N. Foerster et al. “Counterfactual Multi-Agent Policy Gradients”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.1 (2018) (cit. on p. 9).
- [GR01] Yoav Gabriely and Elon Rimon. “Spanning-Tree Based Coverage of Continuous Areas by a Mobile Robot”. In: *Annals of Mathematics and Artificial Intelligence* 31 (2001), pp. 47–79 (cit. on pp. 1, 6).
- [GC13] Enric Galceran and Marc Carreras. “A Survey on Coverage Path Planning for Robotics”. In: *Robotics and Autonomous Systems* 61.12 (2013), pp. 1258–1276. DOI: 10.1016/j.robot.2013.09.004 (cit. on pp. 1, 6).
- [Has+21] Ramin Hasani et al. “Liquid Time-constant Networks”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35.9 (2021), pp. 7657–7666. DOI: 10.1609/aaai.v35i9.16936 (cit. on pp. 1, 19, 20, 35).
- [Has+18] Ramin M. Hasani et al. *Liquid Time-constant Recurrent Neural Networks as Universal Approximators*. 2018. arXiv: 1811.00321 [cs.LG]. URL: <https://arxiv.org/abs/1811.00321> (cit. on pp. 1, 19, 20).
- [HS15] Matthew Hausknecht and Peter Stone. “Deep Recurrent Q-Learning for Partially Observable MDPs”. In: *2015 AAAI Fall Symposium Series*. 2015 (cit. on p. 10).
- [Hen+18] Peter Henderson et al. “Deep Reinforcement Learning That Matters”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018 (cit. on p. 20).
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780 (cit. on pp. 15, 65).
- [Hu+23] Wenjian Hu et al. “Multi-UAV Coverage Path Planning: A Distributed Online Cooperation Method”. In: *IEEE Transactions on Vehicular Technology* 72.9 (2023), pp. 11727–11740. DOI: 10.1109/TVT.2023.3266817 (cit. on p. 13).

- [Lec+19] Mathias Lechner et al. “Designing Worm-inspired Neural Networks for Interpretable Robotic Control”. In: *2019 International Conference on Robotics and Automation (ICRA)*. 2019, pp. 87–94. DOI: 10.1109/ICRA.2019.8793840 (cit. on pp. 13, 35).
- [Lil+16] Timothy P. Lillicrap et al. “Continuous Control with Deep Reinforcement Learning”. In: *International Conference on Learning Representations (ICLR)*. 2016 (cit. on p. 11).
- [Low+17] Ryan Lowe et al. “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 6379–6390 (cit. on pp. 19, 20).
- [Lun+24] Marco Andrés Luna et al. “A Multi-UAV System for Coverage Path Planning Applications with In-flight Re-planning Capabilities”. In: *Journal of Field Robotics* 41.5 (2024), pp. 1480–1497. DOI: 10.1002/rob.22342 (cit. on p. 13).
- [Lyu+23] Mingyang Lyu et al. “Unmanned Aerial Vehicles for Search and Rescue: A Survey”. In: *Remote Sensing* 15.13 (2023), p. 3266. DOI: 10.3390/rs15133266 (cit. on p. 1).
- [MPG24] Antonio Marino, Claudio Pacchierotti, and Paolo Robuffo Giordano. “Liquid-Graph Time-Constant Network for Multi-Agent Systems Control”. In: *2024 IEEE 63rd Conference on Decision and Control (CDC)*. 2024, pp. 793–800. DOI: 10.1109/CDC56724.2024.10886394 (cit. on p. 16).
- [Mni+15] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (2015), pp. 529–533 (cit. on p. 1).
- [Mni+16] Volodymyr Mnih et al. “Asynchronous Methods for Deep Reinforcement Learning”. In: *Proceedings of The 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. PMLR, 2016, pp. 1928–1937 (cit. on p. 1).
- [Sch+15] John Schulman et al. “Trust Region Policy Optimization”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research. PMLR, 2015, pp. 1889–1897 (cit. on p. 11).
- [Sch+17] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347> (cit. on pp. 1, 11, 20, 21, 38).

- [Sch+18] John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2018. arXiv: 1506.02438 [cs.LG]. URL: <https://arxiv.org/abs/1506.02438> (cit. on pp. 12, 38).
- [Sil+16] David Silver et al. “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529.7587 (2016), pp. 484–489 (cit. on p. 1).
- [SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press, 2018 (cit. on p. 1).
- [Tam+17] Ardi Tampuu et al. “Multiagent Deep Reinforcement Learning with Extremely Sparse Rewards”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017 (cit. on p. 19).
- [The+20] Mirco Theile et al. “UAV Coverage Path Planning under Varying Power Constraints using Deep Reinforcement Learning”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 1444–1449. DOI: 10.1109/IROS45743.2020.9340934 (cit. on pp. 13, 24).
- [The+21] Mirco Theile et al. “UAV Path Planning using Global and Local Map Information with Deep Reinforcement Learning”. In: *2021 20th International Conference on Advanced Robotics (ICAR)*. 2021, pp. 539–546. DOI: 10.1109/ICAR53236.2021.9659413 (cit. on pp. 13, 24).
- [The+25] Mirco Theile et al. *Continuous World Coverage Path Planning for Fixed-Wing UAVs using Deep Reinforcement Learning*. 2025. arXiv: 2505.08382 [cs.R0]. URL: <https://arxiv.org/abs/2505.08382> (cit. on pp. 18, 20).
- [Vas+17] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017 (cit. on pp. 38, 65).
- [Wan+25] Qidong Wang et al. “Coverage Path Planning of UAV Cluster Based on Independent PPO in Complex Environment”. In: *2025 37th Chinese Control and Decision Conference (CCDC)*. 2025, pp. 1917–1922. DOI: 10.1109/CCDC65474.2025.11090996 (cit. on p. 13).
- [Wer90] Paul J. Werbos. “Backpropagation Through Time: What It Does and How to Do It”. In: *Proceedings of the IEEE*. Vol. 78. 10. 1990, pp. 1550–1560 (cit. on pp. 20, 35).

Bibliography

- [Wil92] Ronald J. Williams. “Simple Statistical Gradient-following Algorithms for Connectionist Reinforcement Learning”. In: *Machine Learning* 8 (1992), pp. 229–256 (cit. on p. 11).
- [Yam97] Brian Yamauchi. “A Frontier-based Approach for Autonomous Exploration”. In: *1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation*. 1997, pp. 146–151 (cit. on p. 7).

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese Masterarbeit selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe und alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, als solche gekennzeichnet sind, sowie, dass ich die Masterarbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Passau, March 31, 2026



Nauman Ahmed